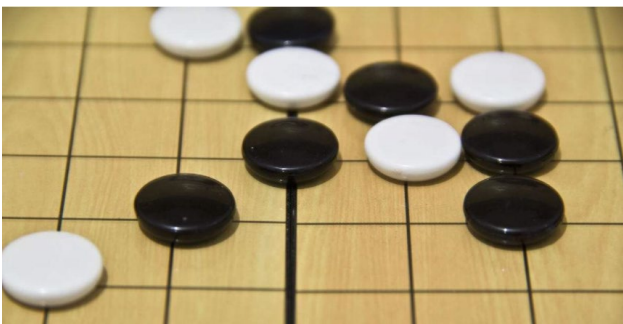


- 5.4.1 启发信息和估价函数
- 5.4.2 A搜索算法
- 5.4.3 A*搜索算法及其特性分析
- 补充：博弈搜索策略



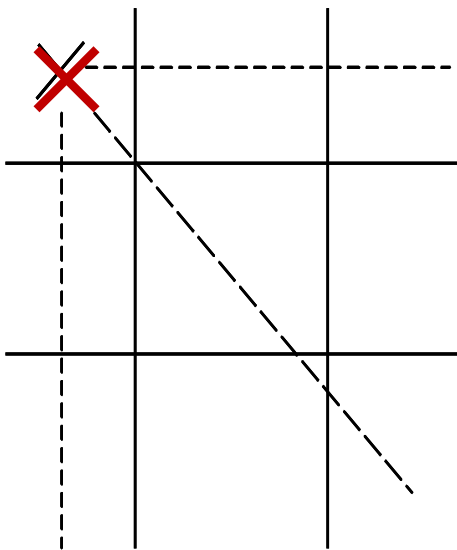
博弈树



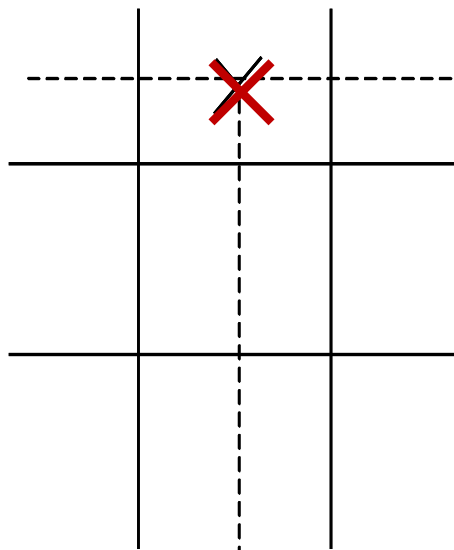
- 例5.6， 一字棋或井字棋（Tic-Tac-Toe游戏）
（http://www.4399.com/flash/50489_1.htm）。在九宫棋盘上，从空棋盘开始，双方轮流在棋盘上摆各自的棋子 × 或 ○（每次一枚），谁先取得三子一线（一行、一列或一条对角线）的结果就取胜。

×	×	×
○	×	
○		○

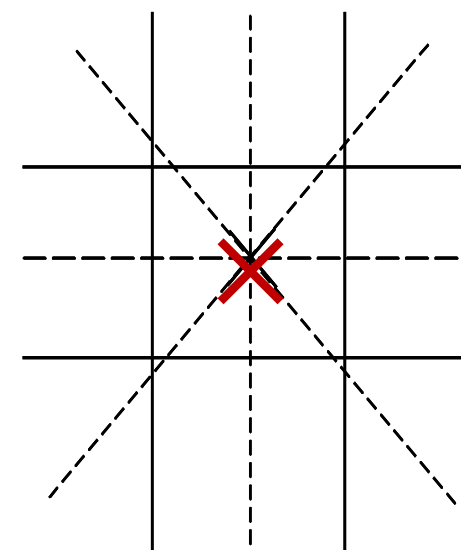
- × 和 ○ 能够在棋盘中摆成的各种不同的棋局就是问题空间中的不同状态。
- 最多的可能走法： $9 \times 8 \times 7 \dots \times 1 = 9! = 362,880$ 种棋局。



赢的几率 ③



赢的几率 ②



赢的几率 ④

棋局的启发值：所有空格都放上X后三子一线的总数 - 所有空格都放上O后三子一线总数

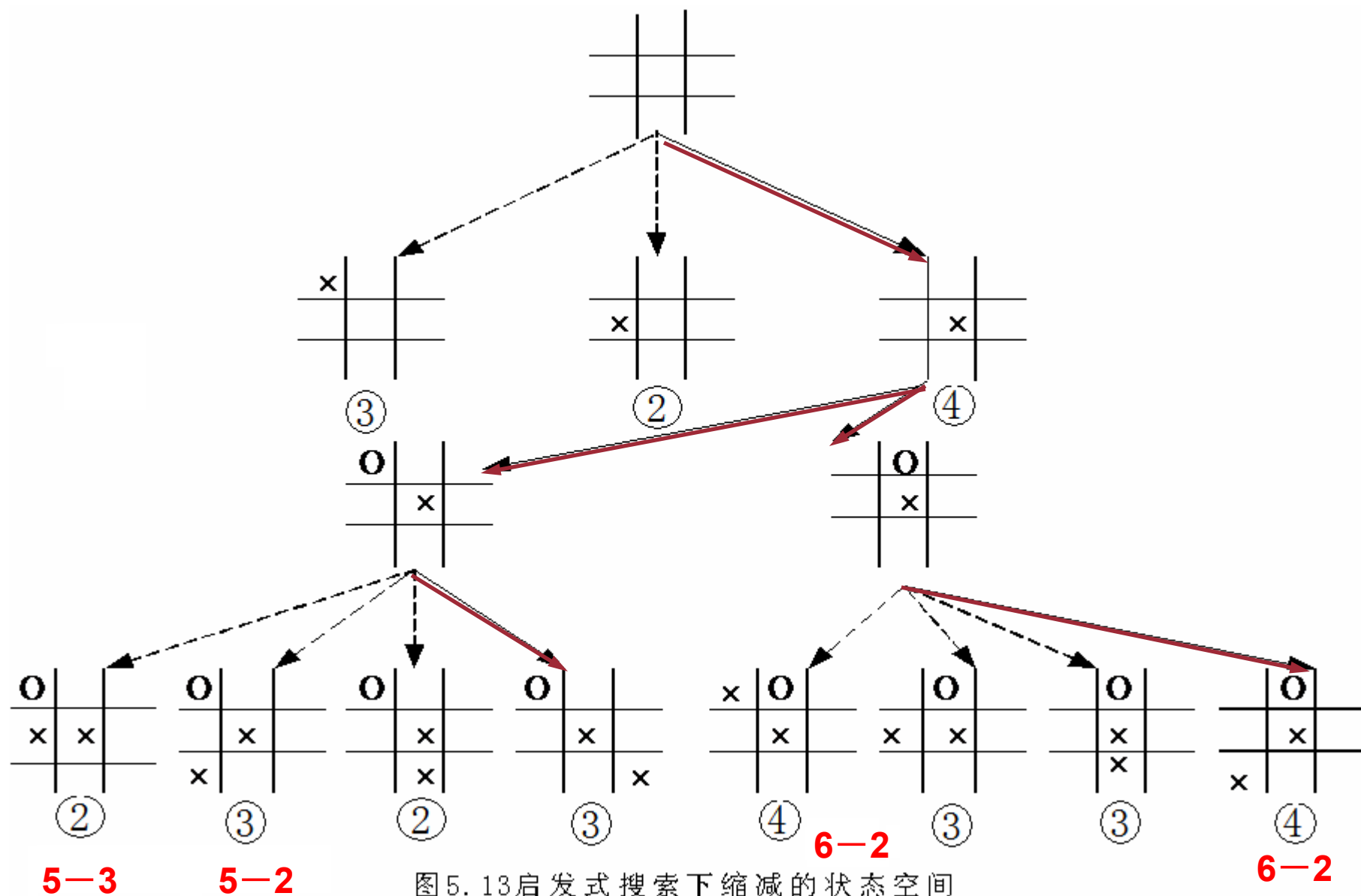
8-5

8-6

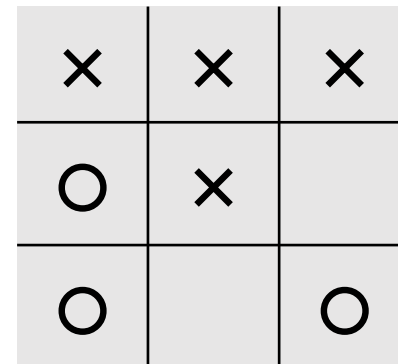
8-4

图5.12 启发式策略的运用

棋局的启发值：所有空格都放上X后三子一线的总数 - 所有空格都放上O后三子一线总数



- 采用启发式策略的一字棋搜索的上限大约为 4.5×9 ，近40种状态，比原来的 $9!$ 大小的状态空间缩小了很多。
- 现实中有许多复杂问题是阶乘或指数级别的，其规模很大，例如：
 - 西洋跳棋是 10^{78} ，国际象棋是 10^{120} ，围棋是 10^{761} 。
 - 假设计算机每步可以搜索一个棋局，用极限并行速度（ 10^{-104} 年/步）来处理，搜索一遍国际象棋的全部棋局就要 10^{16} 年即1亿亿年，而已知的宇宙寿命才100亿年。



4



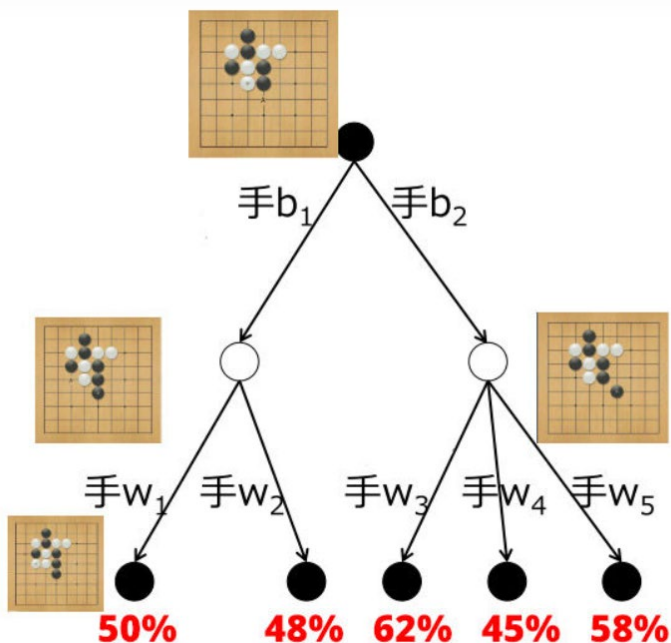
- **极小极大搜索(Minimax Search)**或最小最大搜索：博弈搜索中最为基本的一种让玩家来计算最优策略的方法。
- **Alpha-Beta剪枝搜索(α - β 剪枝搜索)**：一种对最小最大搜索进行改进的算法，即在搜索过程中可剪除无需搜索的分支节点，且不影响搜索结果。
- **蒙特卡洛树搜索(Monte-Carlo Tree Search, MCTS)**：通过采样而非穷举方法来实现搜索。



- 整个博弈过程属于**零和博弈**，即一方的收益必然意味着另一方的损失，博弈双方的收益和损失相加总和永远是零，双方不存在任何合作的可能。
- 博弈双方足够聪明，即每一方在决策时总会选择使自己利益最大化的决策。
- **极小极大搜索(Minimax Search)**: 一种对抗性搜索算法，用于**两人零和完全信息**（**双方都能看到完整游戏状态**）博弈。

正方 (MAX节点, 我方) 从所有子节点中, 选取具有**最大评估值**的节点。

反方 (MIN节点, 对手) 从其所有子节点中, 选取具有**最小评估值**的节点。

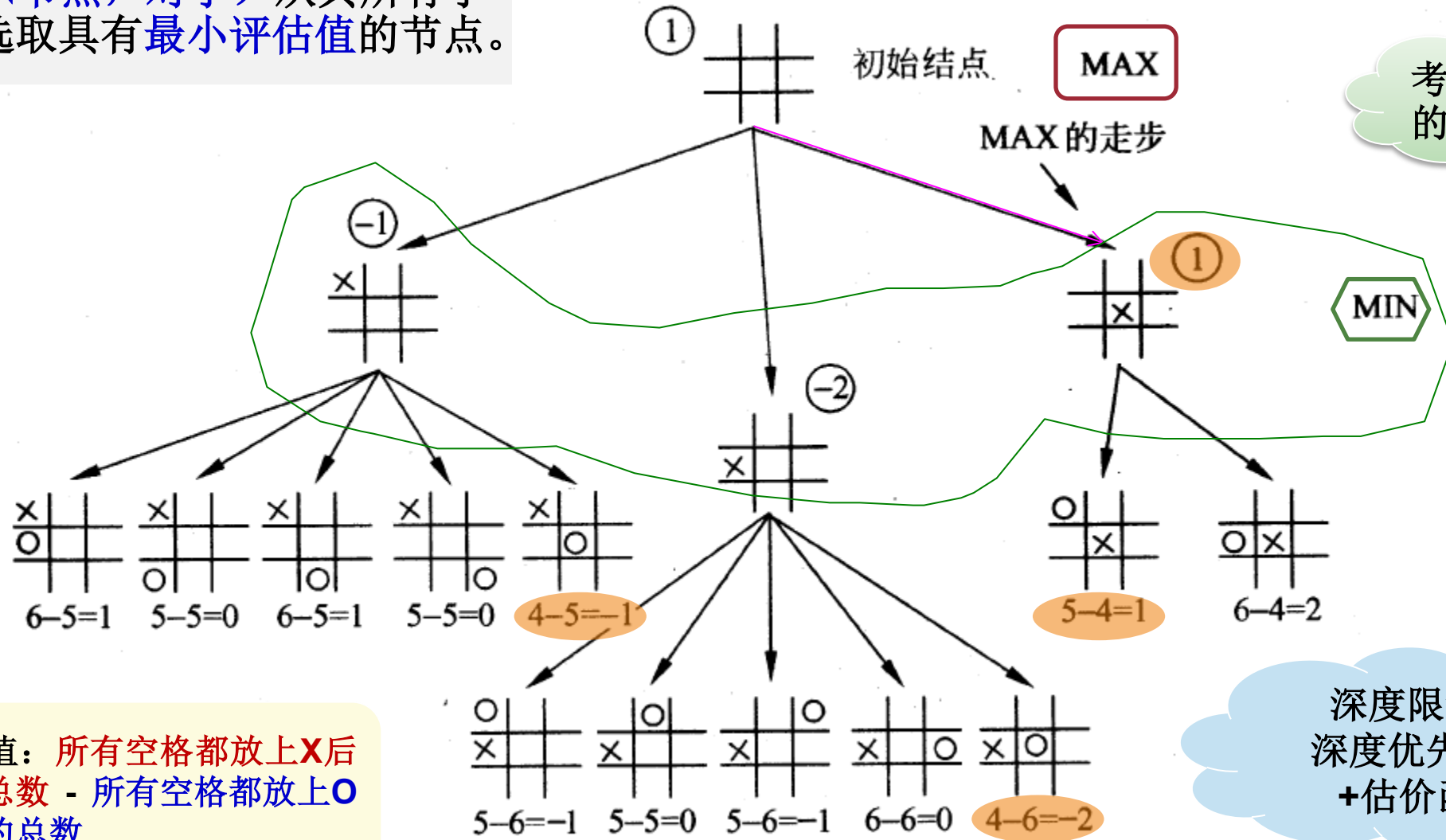


×	×	×
○	×	
○		○

极小极大搜索过程

正方（MAX节点，我方）从所有子节点中，选取具有**最大评估值**的节点。

反方（MIN节点，对手）从其所有子节点中，选取具有**最小评估值**的节点。



棋局的启发值：所有空格都放上X后三子一线的总数 - 所有空格都放上O后三子一线的总数

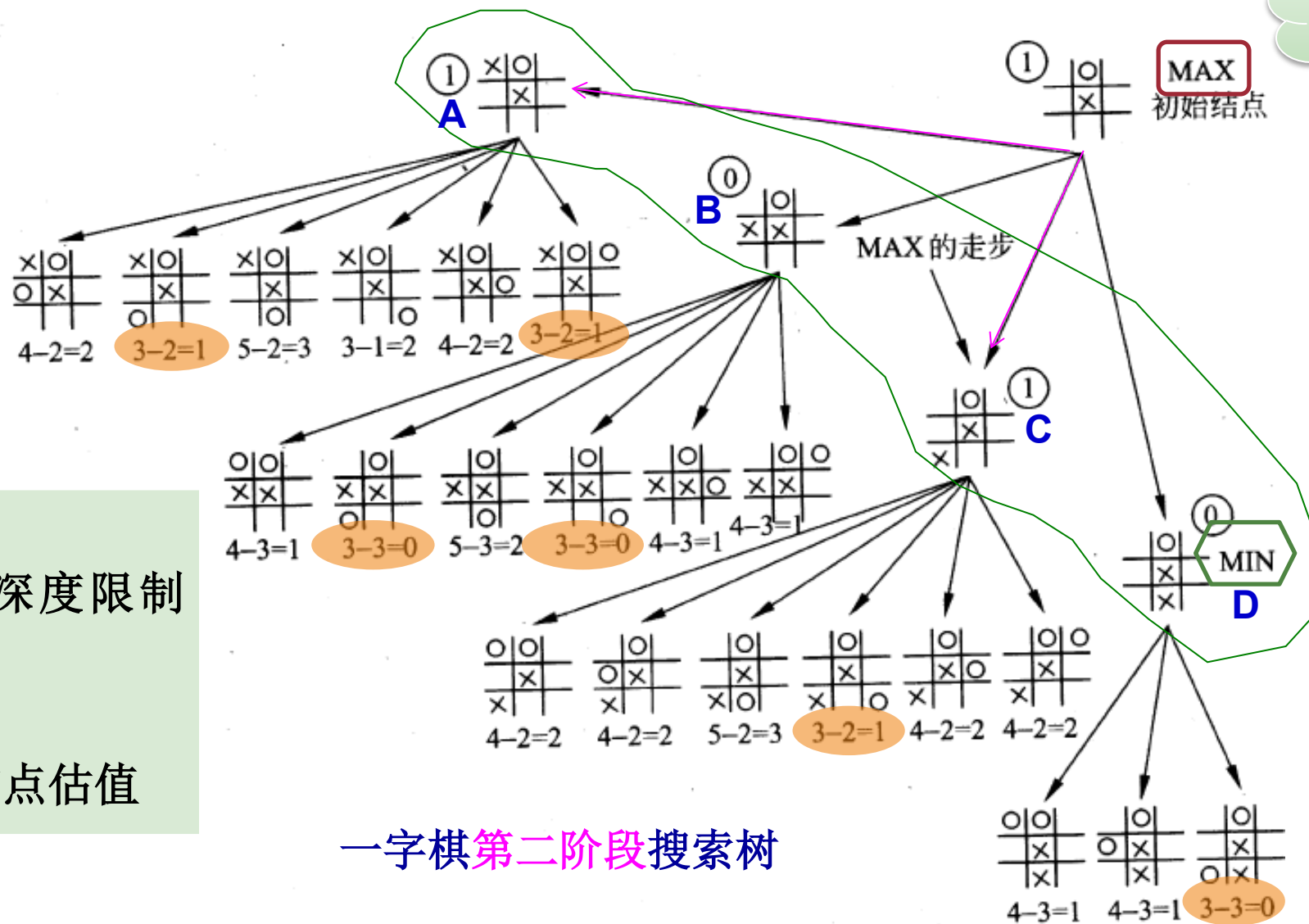
深度限制的
深度优先搜索
+ 估价函数

一字棋第一阶段搜索树

极小极大搜索过程

棋局的启发值：所有空格都放上X后
三子一线的总数 - 所有空格都放上O
后三子一线的总数

考虑走两步
的搜索过程



极大极小算法步骤：

- 1) 生成博弈树（深度限制）
- 2) 评估叶子节点
- 3) 自底向上计算节点估值
- 4) 选择最优动作

一字棋第二阶段搜索树

- 优点:

- 算法是一种简单有效的博弈搜索手段
- 在对手也“尽力而为”前提下，算法可返回最优结果

- 缺点:

- 如果搜索树极大，则无法在有效时间内返回结果
- 对于非终局状态，需要设计好的评估函数

- 改善:

- 使用Alpha-Beta剪枝算法来减少搜索节点，降低搜索的宽度
- 对节点进行采样、而非逐一搜索 (例如蒙特卡洛树搜索，通过随机模拟计算节点估值，不需要预先设计评估函数)

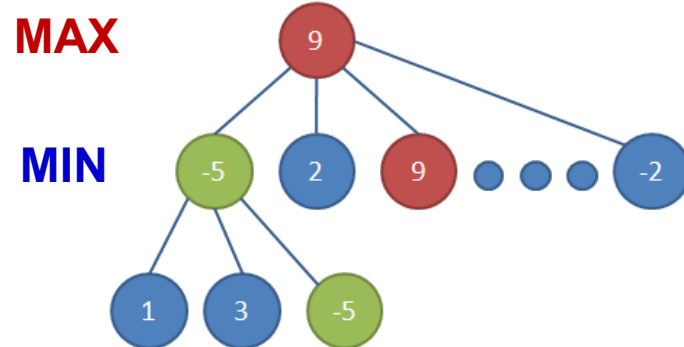
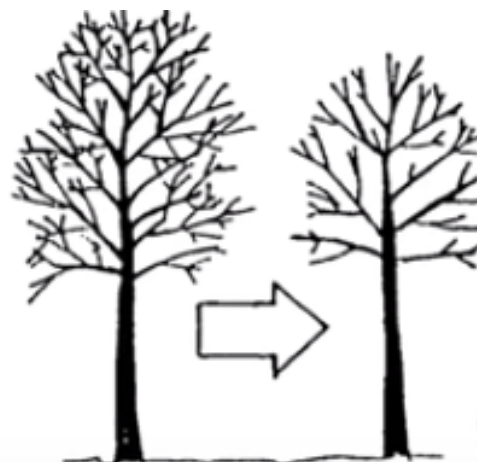


图-极大极小搜索



α - β 剪枝法 (20世纪50年代, 约翰. 麦卡锡)

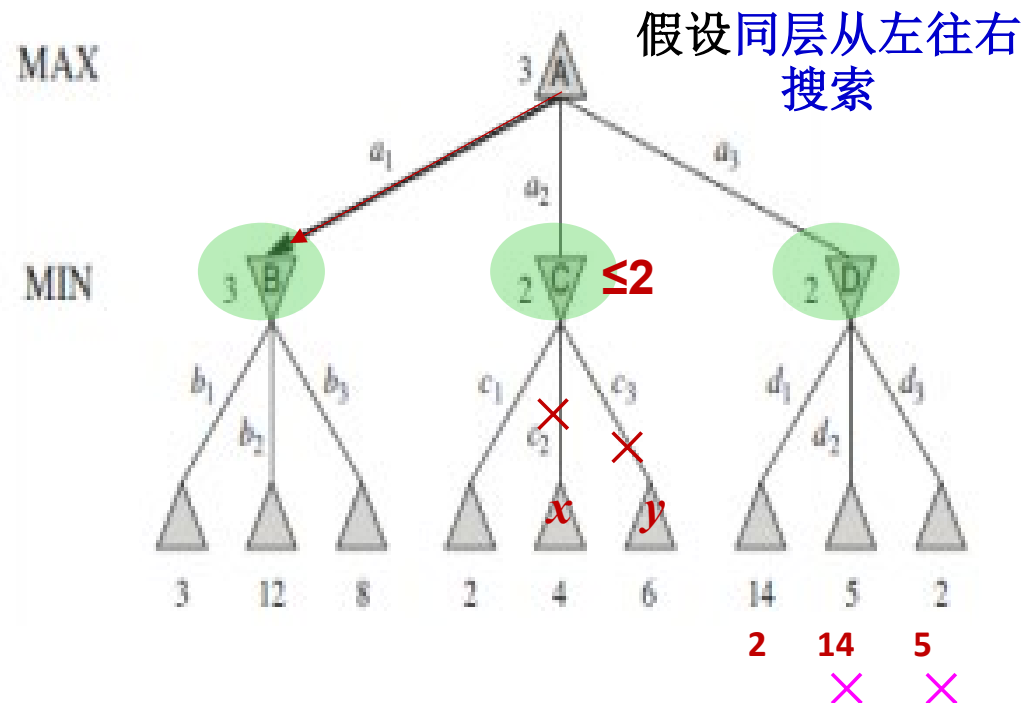
在极小化极大算法 (minimax算法) 中减少所搜索的搜索树节点数。该算法和极小化极大算法所得结论相同, 但剪去了不影响最终结果的搜索分枝。

$$\begin{aligned}
 & MINIMAX(root) \\
 &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\
 &= \max(3, \min(2, x, y), 2) \\
 &= \max(3, z, 2) = 3
 \end{aligned}$$

where $z = \min(2, x, y) \leq 2$
 可以看出: 根节点 (即 MAX 选手) 的选择与 x 和 y 两个值无关 (因此, x 和 y 可以被剪枝去除)

深度限制的深度优先搜索
 + 估价函数 + 剪枝

需要注意的是, 剪枝的效果与树节点的访问顺序有关。



图中MIN所在的节点C下属分支4和6与根节点最终优化决策的取值无关, 可不访问

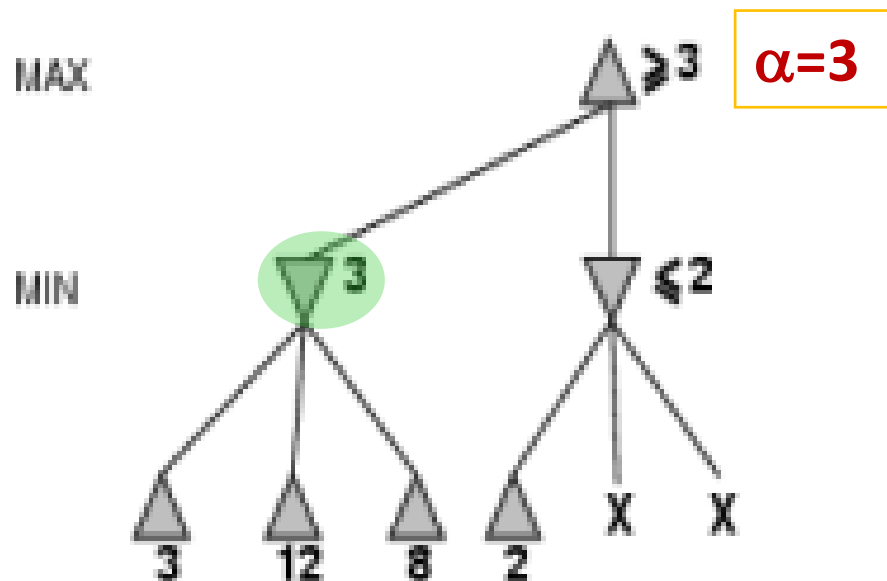
■ MAX节点的评估下限值 α

作为正方出现的MAX节点，假设它的MIN子节点有N个，那么当它的**第一个MIN子节点的评估值为 α** 时，则对于其它的子节点，**如果有高过 α 的，就取那最高的值作为该MAX节点的评估值**；如果没有，则该MAX节点的评估值为 α 。

总之，**该MAX节点的评估值不会低于 α** ，这个 α 就称为该MAX节点的评估下限值。

◆极大（MAX）节点的下界（MAX玩家至少能得到的分数）为 α

◆极小（MIN）节点的上界（MIN玩家至多能得到的分数）为 β



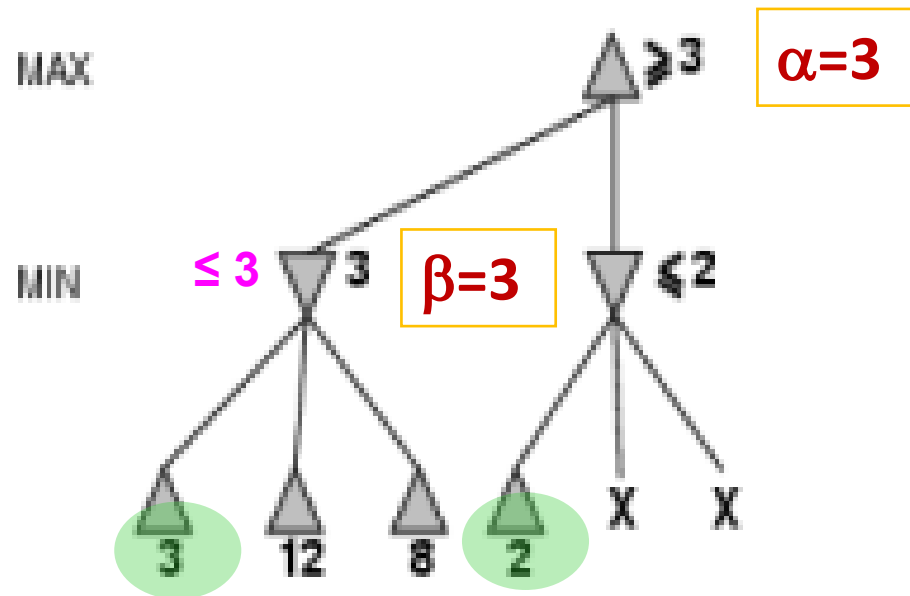
- MIN节点的评估上限值 β

作为反方出现的MIN节点，假设它的MAX子节点有N个，那么当它的第一个MAX子节点的评估值为 β 时，则对于其它子节点，如果有低于 β 的，就取那个低于 β 的值作为该MIN节点的评估值；如果没有，则该MIN节点的评估值取 β 。

总之，该MIN节点的评估值不会高过 β ，这个 β 就称为该MIN节点的评估上限值。

◆极大 (MAX) 节点的下界 (MAX玩家至少能得到的分数) 为 α

◆极小 (MIN) 节点的上界 (MIN玩家至多能得到的分数) 为 β

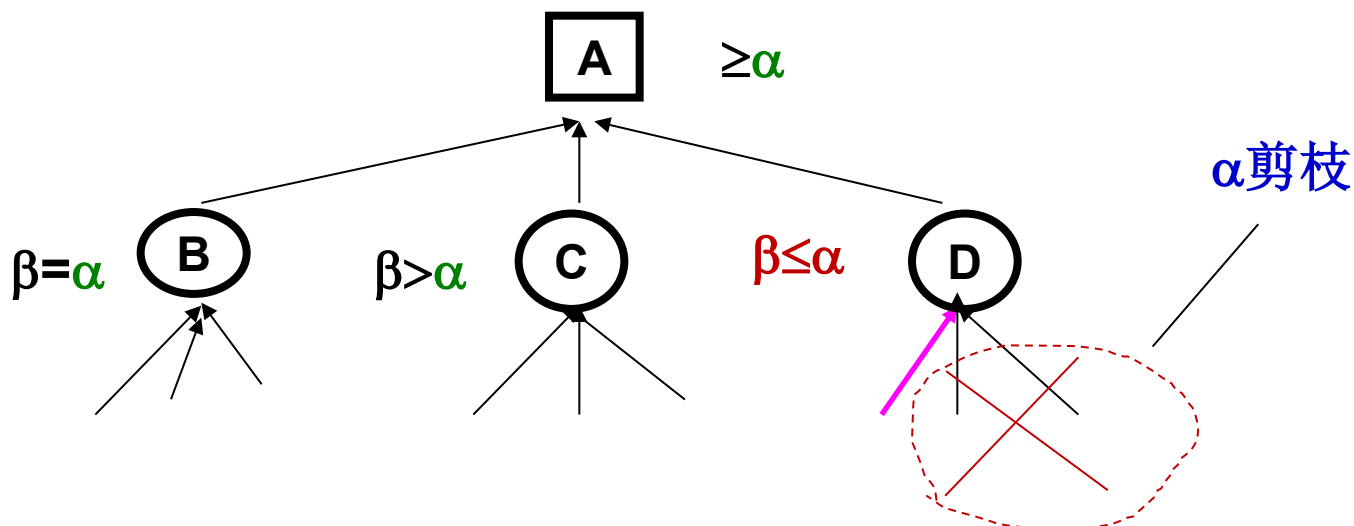


- α 剪枝

设**MAX**节点的下限为 α ，则其所有的**MIN**子节点中，其评估值的 β 上限小于等于 α 的节点，其以下部分的搜索都可以停止了，即对这部分节点进行了 α 剪枝。

MAX节点

MIN节点

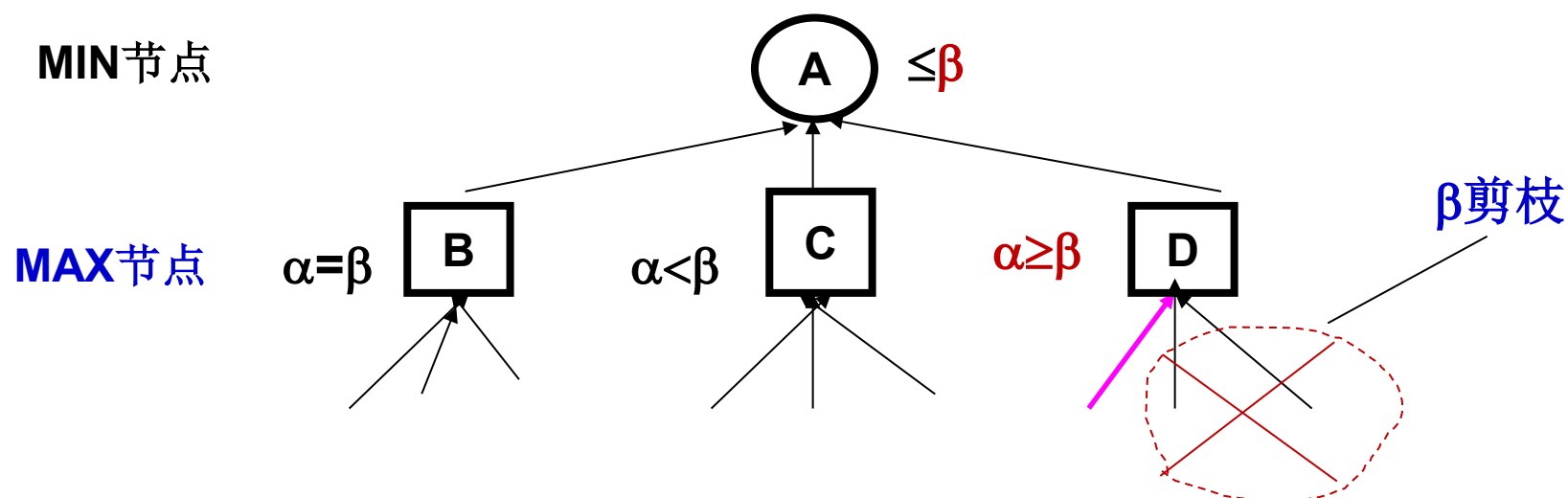


- ◆极大 (**MAX**) 节点的下界为 α
- ◆极小 (**MIN**) 节点的上界为 β

当一个 Min 节点的 β 值 $\leq$ 任何一个祖先节点的 α 值时，剪掉该节点的后继子节点

■ β 剪枝

设MIN节点的上限为 β ，则其所有的MAX子节点中，其评估值的 α 下限大于等于 β 的节点，其以下部分的搜索都可以停止了，即对这部分节点进行了 β 剪枝。



◆极大 (**MAX**) 节点的下界为 α

◆极小 (**MIN**) 节点的上界为 β

当一个 Max 节点的 α 值 $\geq$ 任何一个祖先节点的 β 值时，剪掉该节点的后续子节点

- **MAX节点**的下界为 α
- **MIN节点**的上界为 β
- 剪枝的条件:

- **MIN节点**: 其 β 值 $\leq$ 祖先节点的 α 值时, α 剪枝
- **MAX节点**: 其 α 值 $\geq$ 祖先节点的 β 值时, β 剪枝

假设节点N的收益为 $\text{reward}(N)$, 对于 $\alpha \leq \text{reward}(N) \leq \beta$:
 若 $\alpha \leq \beta$ 则 $\text{reward}(N)$ 有解。
 若 $\alpha > \beta$ 则 $\text{reward}(N)$ 无解, 可剪枝

Alpha值(α)	MAX节点目前得到的最高收益
Beta值(β)	MIN节点目前可给对手的最小收益
α 和 β 的值初始化分别设置为 $-\infty$ 和 ∞	



每个节点有两个值, 分别是 α 和 β 。节点的 α 和 β 值在搜索过程中不断变化。其中, α 从负无穷大($-\infty$)逐渐增加、 β 从正无穷大(∞)逐渐减少。如果一个节点中 $\alpha > \beta$, 则该节点的后续节点可剪枝。

- **MAX节点**的下界为 α
- **MIN节点**的上界为 β
- 剪枝的条件:

假设节点N的收益为 $\text{reward}(N)$, 对于 $\alpha \leq \text{reward}(N) \leq \beta$:
 若 $\alpha \leq \beta$ 则 $\text{reward}(N)$ 有解。
 若 $\alpha > \beta$ 则 $\text{reward}(N)$ 无解, 可剪枝

- **MIN节点**: 其 β 值 $\leq$ 祖先节点的 α 值时, α 剪枝
- **MAX节点**: 其 α 值 $\geq$ 祖先节点的 β 值时, β 剪枝

Alpha值(α)	MAX节点目前得到的最高收益
Beta值(β)	MIN节点目前可给对手的最小收益

α 和 β 的值初始化分别设置为 $-\infty$ 和 ∞



每个节点有两个值, 分别是 α 和 β 。节点的 α 和 β 值在搜索过程中不断变化。其中, α 从负无穷大($-\infty$)逐渐增加、 β 从正无穷大(∞)逐渐减少。如果一个节点中 $\alpha > \beta$, 则该节点的后续节点可剪枝。

- **MAX节点**的下界为 α
- **MIN节点**的上界为 β
- 剪枝的条件:

假设节点N的收益为 $\text{reward}(N)$, 对于 $\alpha \leq \text{reward}(N) \leq \beta$:
 若 $\alpha \leq \beta$ 则 $\text{reward}(N)$ 有解。
 若 $\alpha > \beta$ 则 $\text{reward}(N)$ 无解, 可剪枝

- **MIN节点**: 其 β 值 $\leq$ 祖先节点的 α 值时, α 剪枝
- **MAX节点**: 其 α 值 $\geq$ 祖先节点的 β 值时, β 剪枝

Alpha值(α)	MAX节点目前得到的最高收益
Beta值(β)	MIN节点目前可给对手的最小收益

α 和 β 的值初始化分别设置为 $-\infty$ 和 ∞



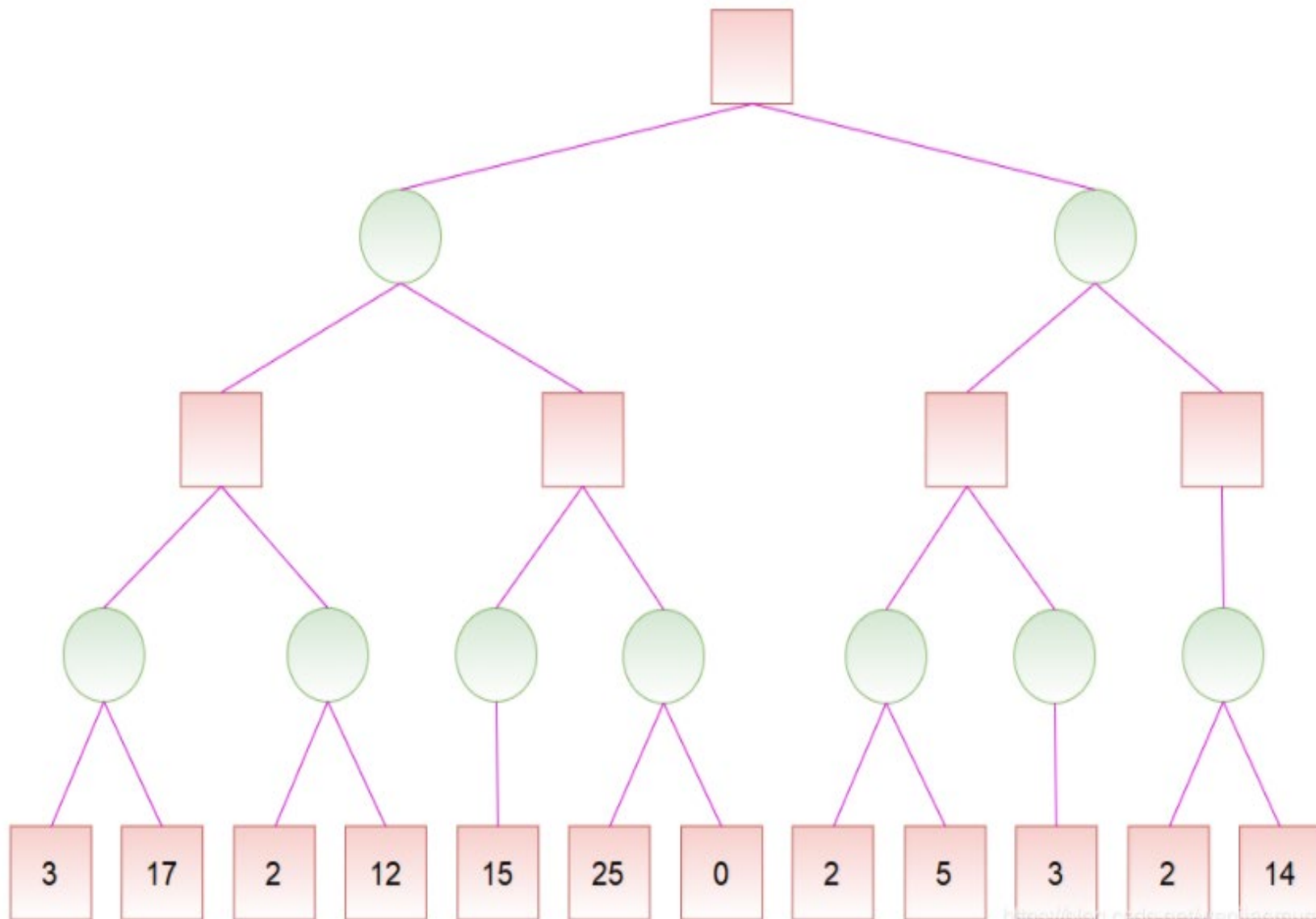
每个节点有两个值, 分别是 α 和 β 。节点的 α 和 β 值在搜索过程中不断变化。其中, α 从负无穷大($-\infty$)逐渐增加、 β 从正无穷大(∞)逐渐减少。如果一个节点中 $\alpha > \beta$, 则该节点的后续节点可剪枝。



1) 构建博弈树:

- 生成博弈树(深度限制)
- 计算叶子节点估值

深度限制的深度优先
搜索（假设深度为4、
同层从左往右搜索）
+估价函数+剪枝





Max节点 : 取大, 更新 α 值

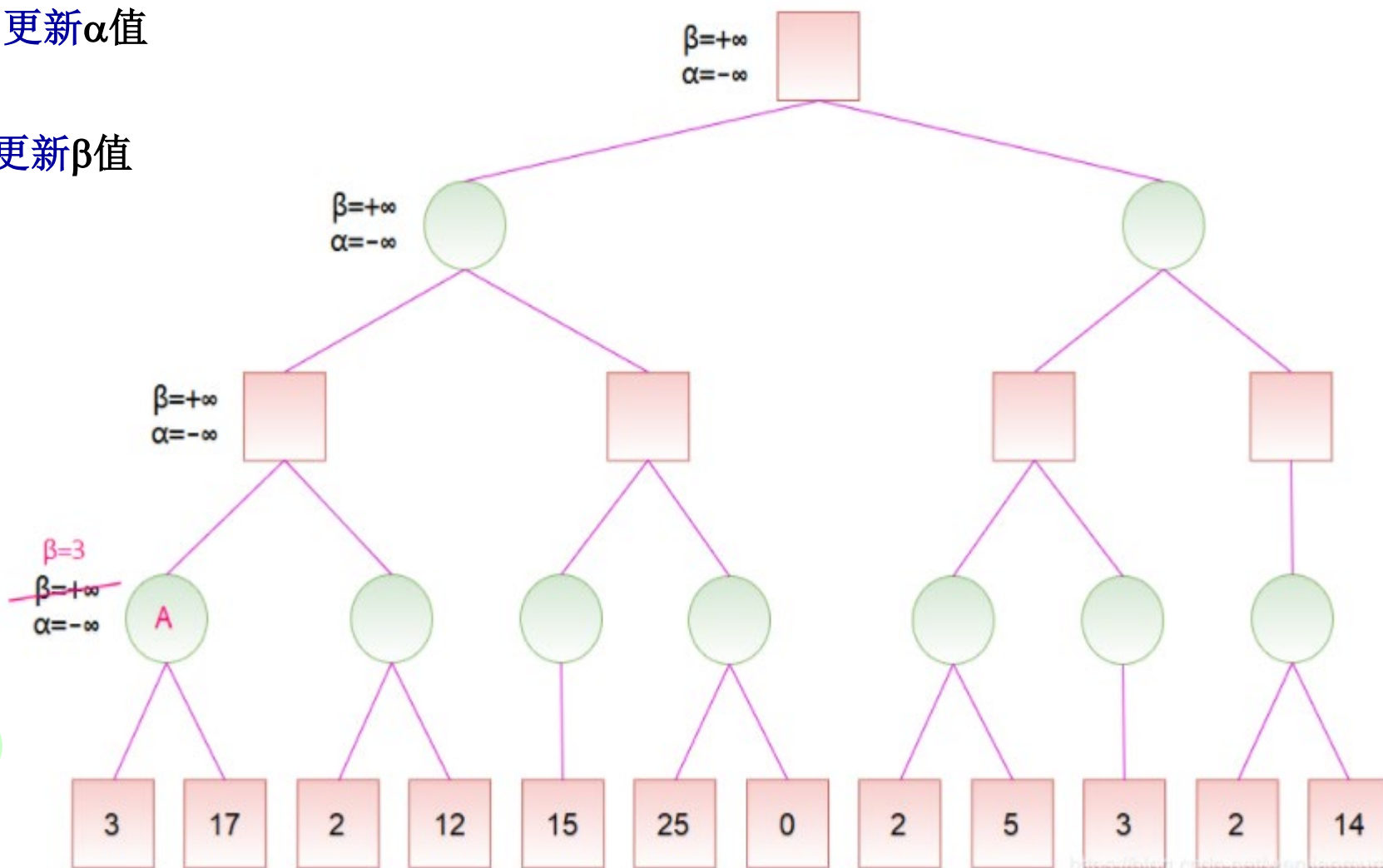


Min节点 : 取小, 更新 β 值

2) 搜索与剪枝:

- α 、 β 更新或剪枝
- 计算节点估值

深度限制的深度优先
搜索 (假设深度为4、
同层从左往右搜索)
+估价函数+剪枝





Max节点：取大，更新 α 值

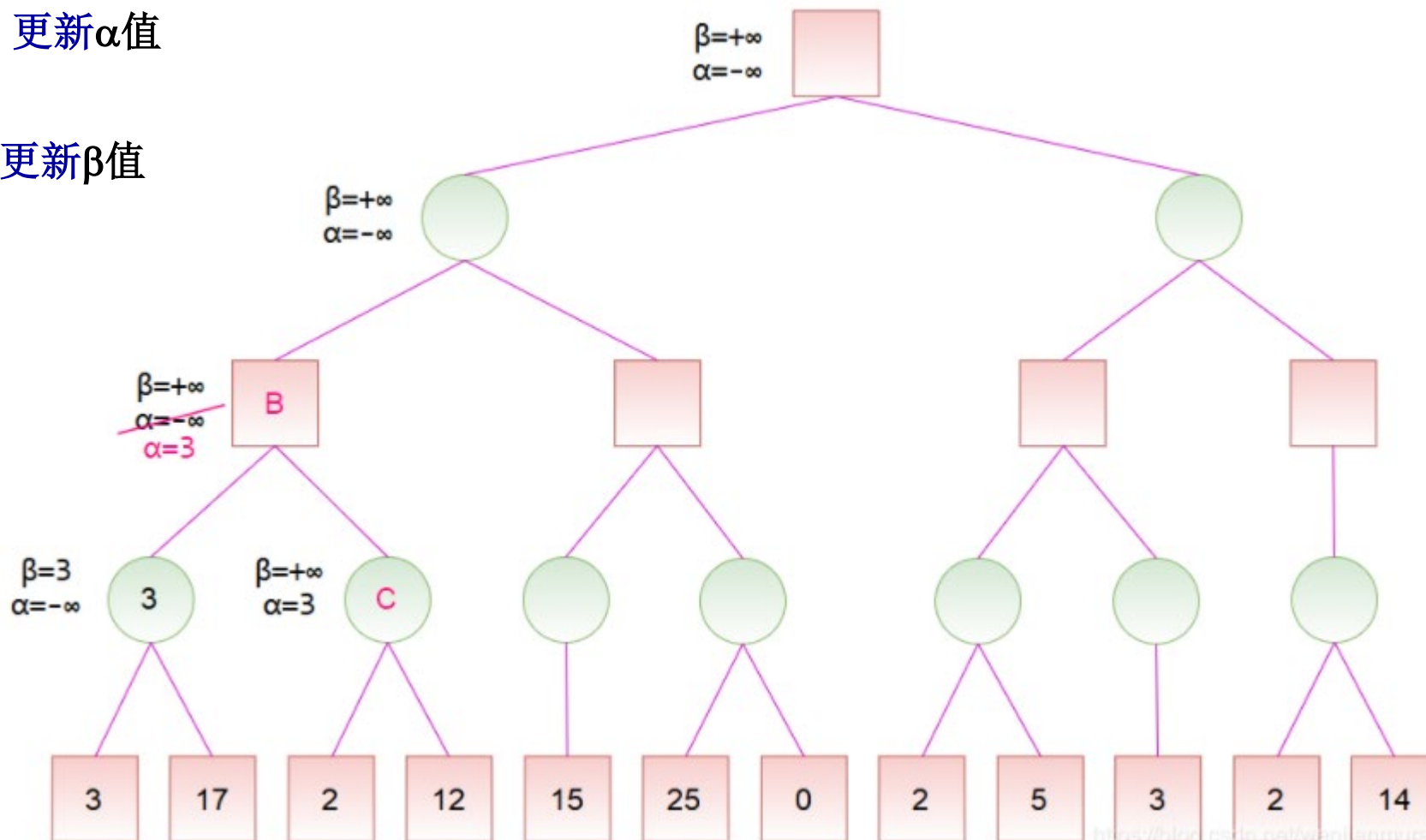


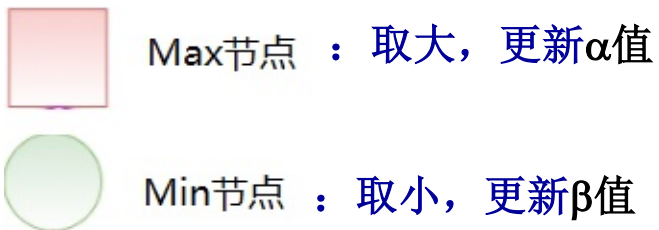
Min节点：取小，更新 β 值

2) 搜索与剪枝：

- α 、 β 更新或剪枝
- 计算节点估值

深度限制的深度优先
搜索（假设深度为4、
同层从左往右搜索）
+估价函数+剪枝

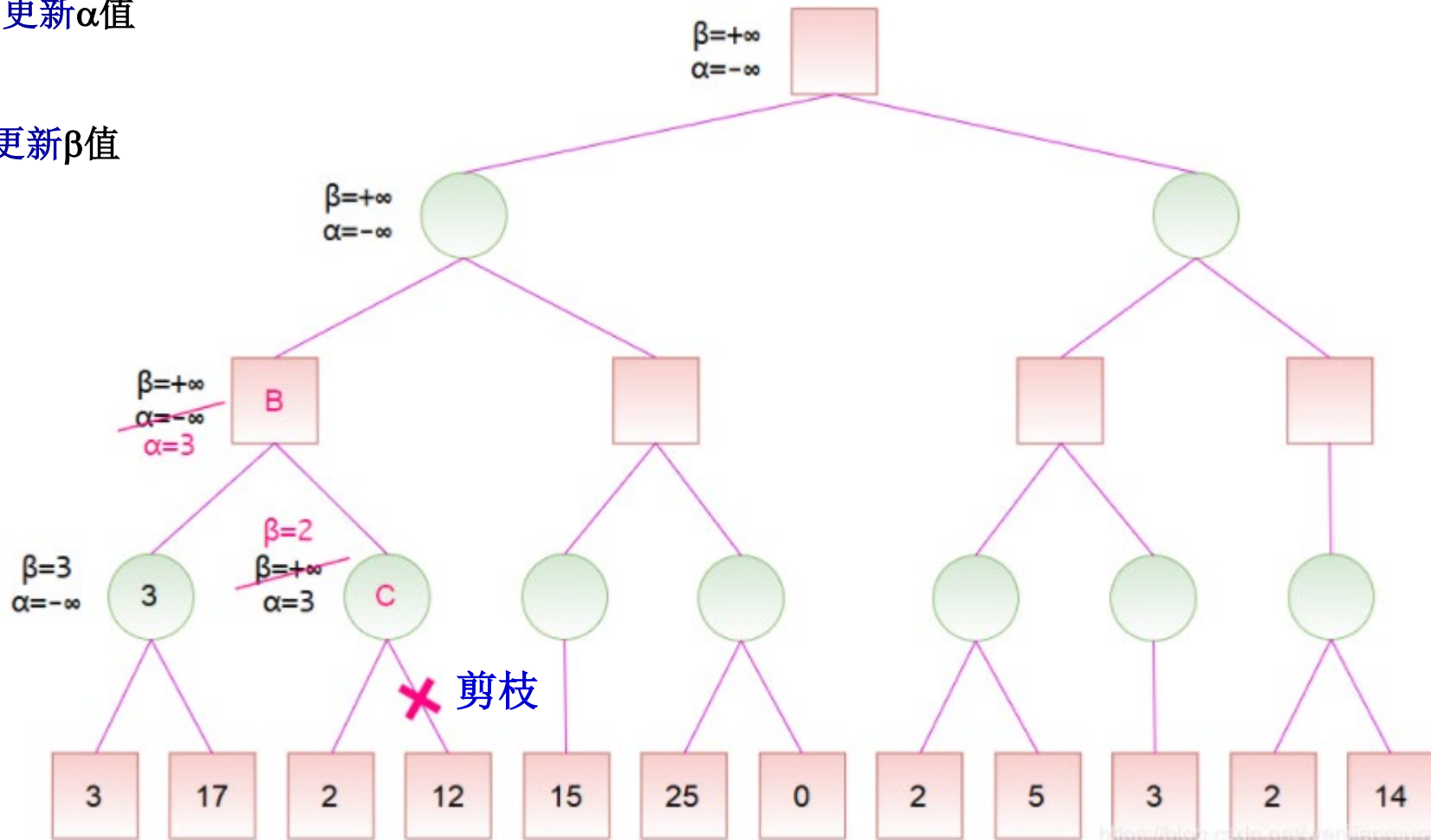




2) 搜索与剪枝:

- α 、 β 更新或剪枝
- 计算节点估值

深度限制的深度优先搜索（假设深度为4、同层从左往右搜索）
+估价函数+剪枝



- **Min 节点**: 其 β 值 $\leq$ 祖先节点的 α 值时, 剪掉该节点的后续子节点, α 剪枝
- **Max 节点**: 其 α 值 $\geq$ 祖先节点的 β 值时, 剪掉该节点的后续子节点, β 剪枝



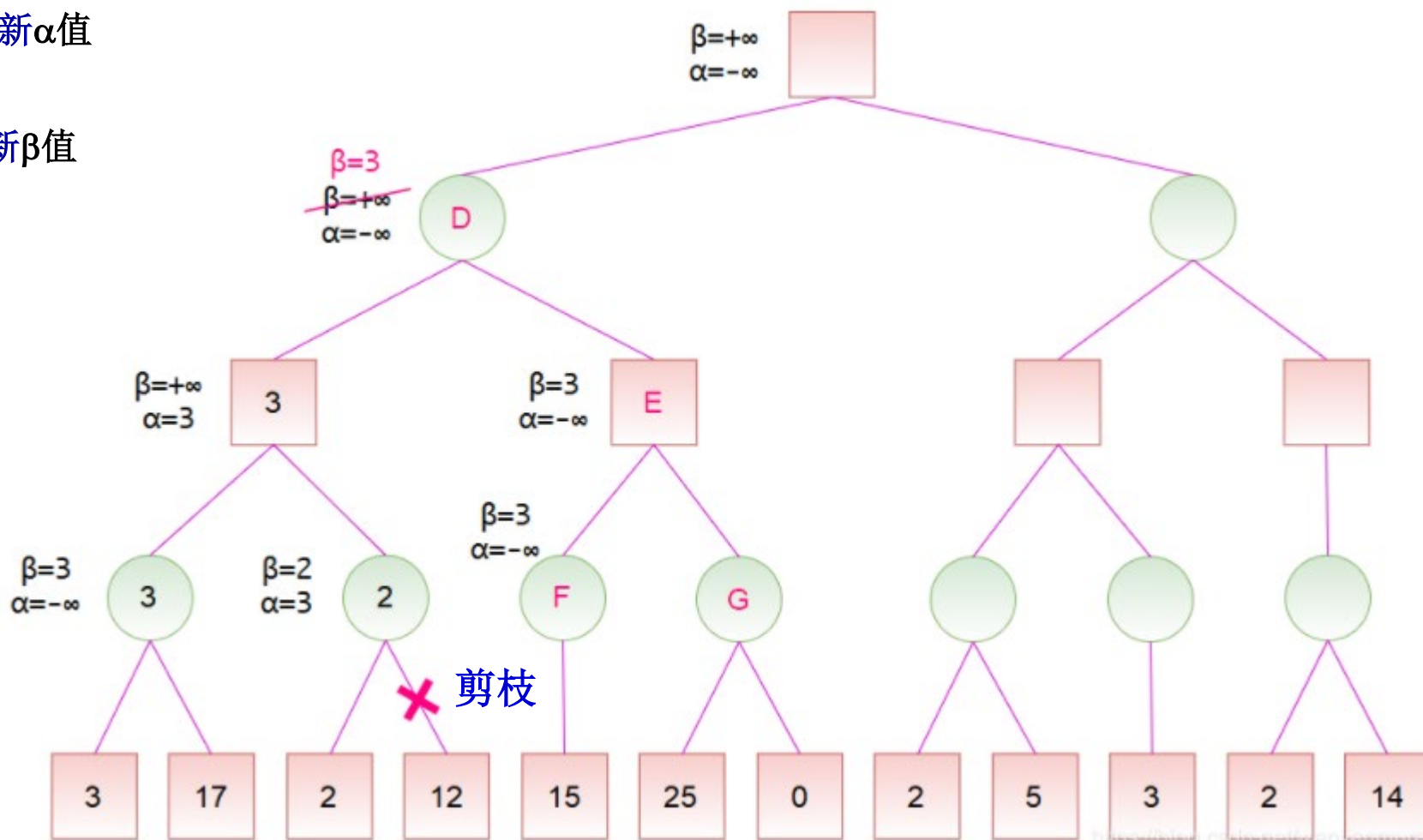
Max节点：取大，更新 α 值



Min节点：取小，更新 β 值

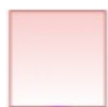
2) 搜索与剪枝：

- α 、 β 更新或剪枝
- 计算节点估值



深度限制的深度优先
搜索（假设深度为4、
同层从左往右搜索）
+估价函数+剪枝

- **Min 节点**：其 β 值 $\leq$ 祖先节点的 α 值时，剪掉该节点的后续子节点， α 剪枝
- **Max 节点**：其 α 值 $\geq$ 祖先节点的 β 值时，剪掉该节点的后续子节点， β 剪枝



Max节点 : 取大, 更新 α 值

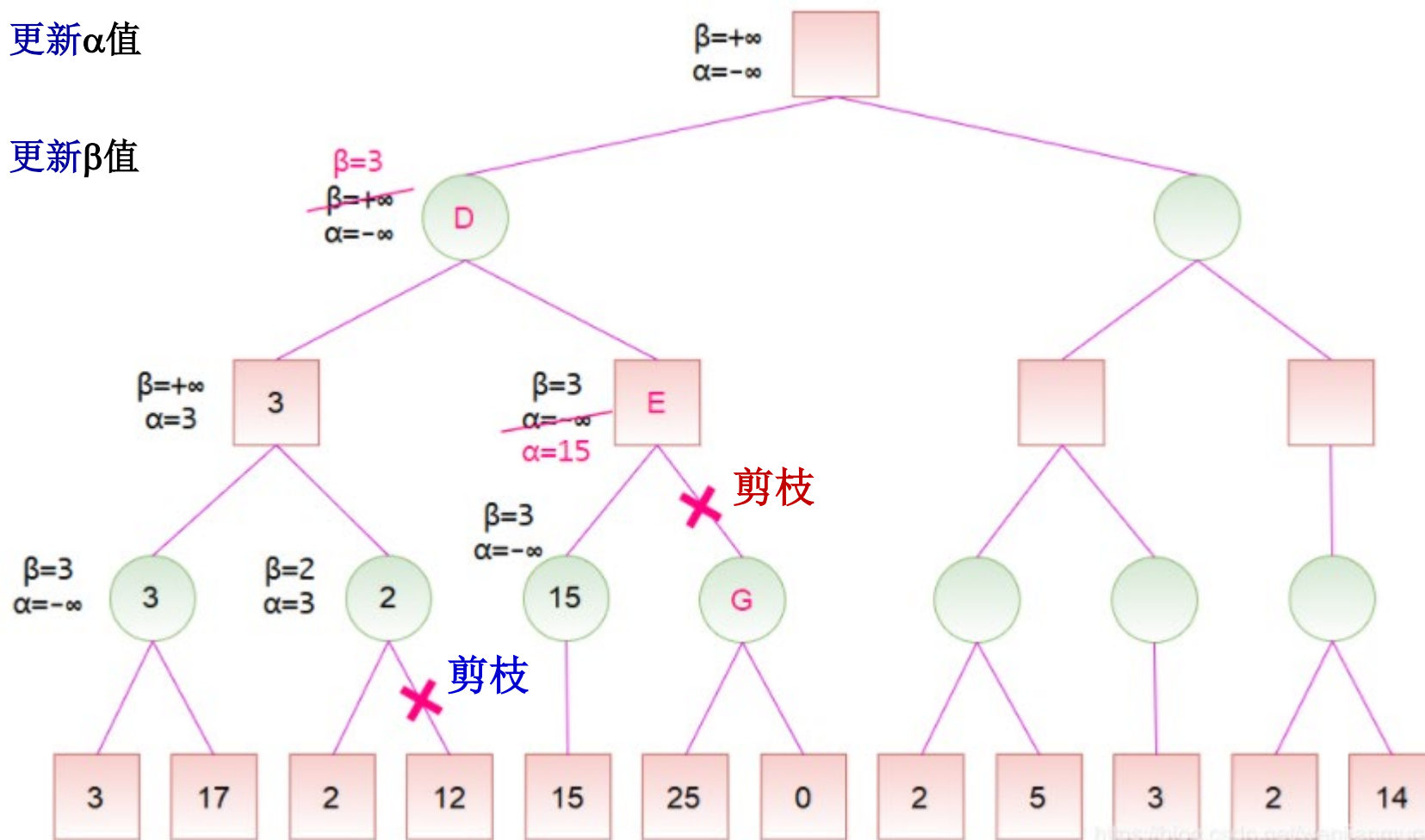


Min节点 : 取小, 更新 β 值

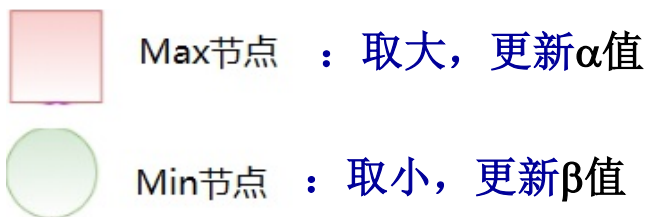
2) 搜索与剪枝:

- α 、 β 更新或剪枝
- 计算节点估值

深度限制的深度
优先搜索 (假设
深度为4、同层从
左往右搜索) + 估
价函数 + 剪枝



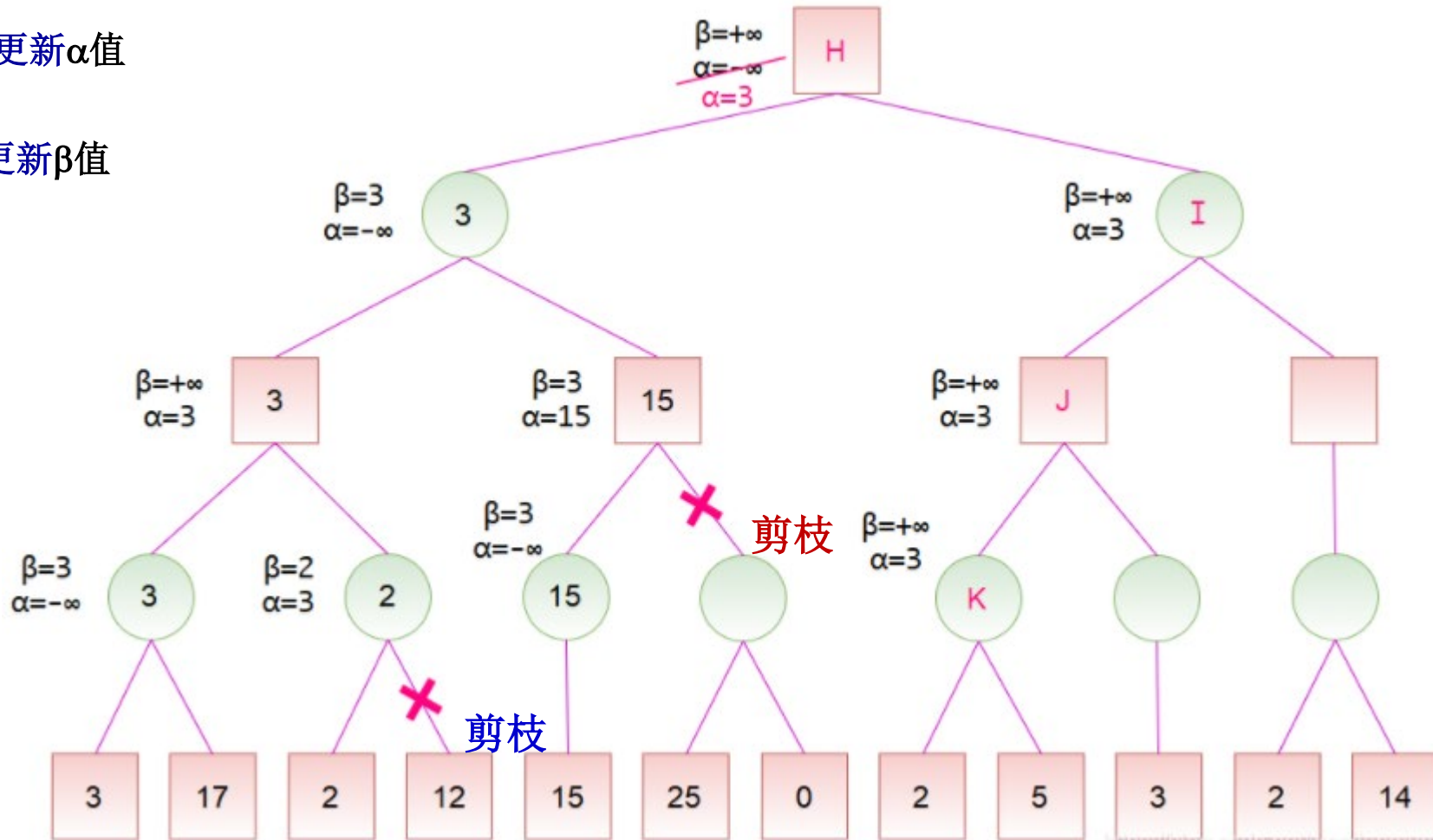
- **Min 节点:** 其 β 值 $\leq$ 祖先节点的 α 值时, 剪掉该节点的后续子节点, α 剪枝
- **Max 节点:** 其 α 值 $\geq$ 祖先节点的 β 值时, 剪掉该节点的后续子节点, β 剪枝



2) 搜索与剪枝:

- α 、 β 更新或剪枝
- 计算节点估值

深度限制的深度优先
 搜索 (假设深度为4、
 同层从左往右搜索)
 +估价函数+剪枝

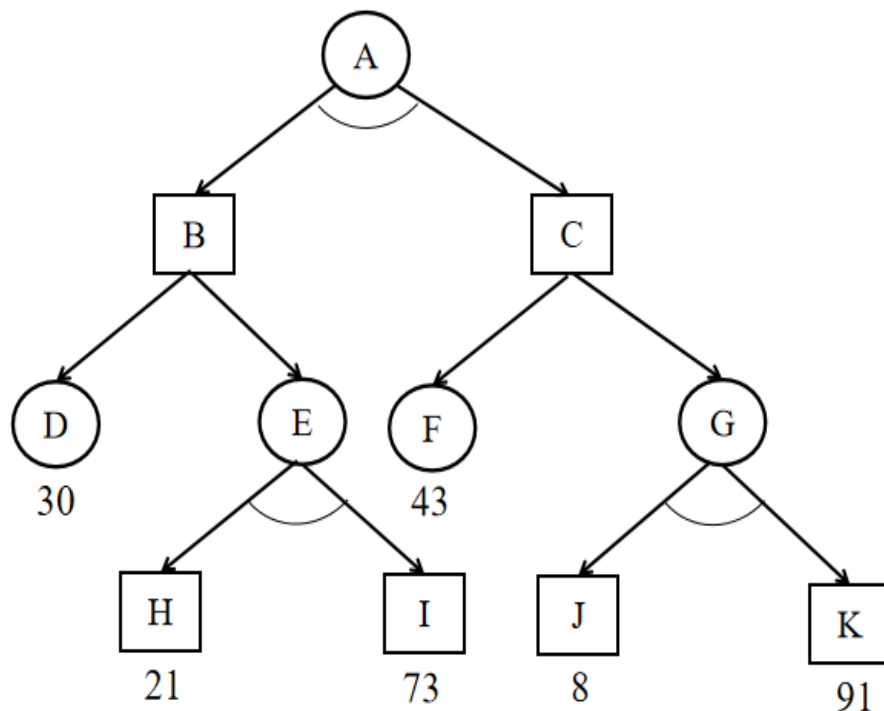


请画出下图最后的Alpha-Beta 剪枝图 (从左到右搜索)

- Min 节点: 其 β 值 $\leq$ 祖先节点的 α 值时, 剪掉该节点的后续子节点, α 剪枝
- Max 节点: 其 α 值 $\geq$ 祖先节点的 β 值时, 剪掉该节点的后续子节点, β 剪枝

下图所示的博弈树（方框表示MAX节点，圆框表示MIN节点），其中最下面的数字为假设的估值，要求：

- 1) 利用 α - β 剪枝技术剪去不必要的分枝（在节点及边上直接加注释和删除标记），同时按极小极大原则计算非叶节点的估值（估值写在非叶节点的旁边），剪掉的节点不需要计算估值；
- 2) 确定节点A最终走出的棋步，即下一步的节点。



Alpha-Beta 剪枝步骤：

- 1) 构建博弈树：
 - 生成博弈树
 - 计算叶子节点估值
- 2) 搜索与剪枝：
 - α 、 β 更新或剪枝
 - 计算节点估值
- 3) 选择最优动作

- 对棋局如何打分是 α - β 剪枝算法中非常关键的内容。
- 深蓝采用规则的方法对棋局打分：对不同的棋子按照重要程度给予不同的分数，还考虑棋子的位置赋予不同的权重、棋子之间的联系等。
- 对于国际象棋，进入残局后，棋子的多少可能就决定了胜负。
- 如果不采用 α - β 剪枝算法，要达到深蓝的下棋水平，每步棋需搜索17年——许峰雄博士



- 剪枝本身不影响算法输出结果，但可以减小搜索宽度
- 节点先后次序会影响剪枝效率
- 如果节点次序“恰到好处”，Alpha-Beta剪枝的时间复杂度为 $O(b^{m/2})$
极小极大搜索的时间复杂度为 $O(b^m)$ ， m 是游戏树的最大深度，在每个节点存在 b 个有效走法

- **极小极大搜索(Minimax Search)**或最小最大搜索：博弈搜索中最为基本的一种让玩家来计算最优策略的方法。
- **Alpha-Beta剪枝搜索(α - β 剪枝搜索)**：一种对最小最大搜索进行改进的算法，即在搜索过程中可剪除无需搜索的分支节点，且不影响搜索结果。
- **蒙特卡洛树搜索(Monte-Carlo Tree Search, MCTS)**：通过随机采样而非穷举方法来实现搜索，特别适用于博弈游戏（如围棋）和其他决策过程。



蒙特卡洛树搜索是一种通过在决策空间中**随机采样**来构建搜索树的方法。它**不需要评估函数**，而是通过多次模拟（simulations）来估计每个行动的价值。模拟通常从当前状态开始，通过随机选择行动直到达到终止状态，然后根据结果（如赢/输）来更新搜索树中的节点统计信息。

对于状态空间巨大、分支因子高的问题（如围棋： 10^{170} 状态空间，分支因子~250），随机采样提供了一种"不求精确，但求统计有效"的解决方案

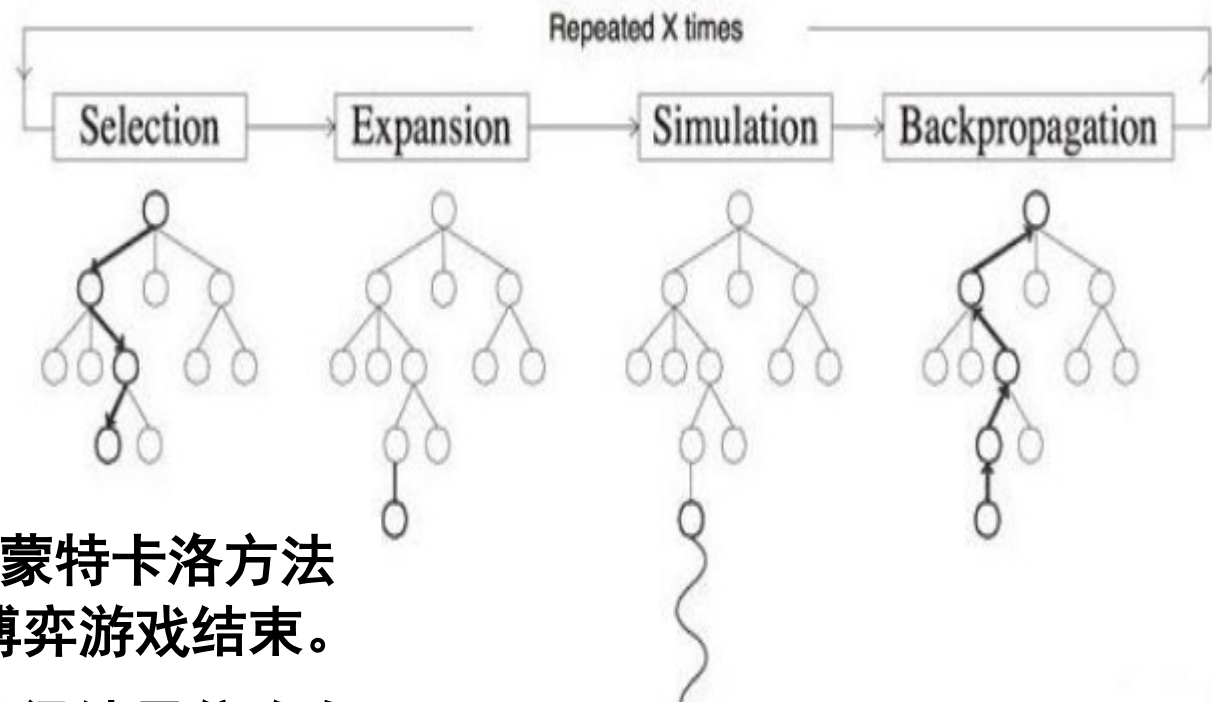
蒙特卡洛树搜索 (MCTS)：是一种通过随机模拟（采样）来近似评估节点价值的搜索方法，在**UCB1** (Upper Confidence Bound, 上限置信区间) 基础上发展出来的一种解决多轮序贯博弈问题的策略。

1. **选择**(Selection): 从根节点开始，递归地选择子节点，直至到达一个叶子节点。

2. **扩展**(Expansion): 向选定的点添加一个或多个子节点。

3. **模拟**(Simulation): 对扩展出的节点用蒙特卡洛方法进行随机模拟，即随机选择行动直到博弈游戏结束。

4. **反传**(Backpropagation): 根据模拟所得结果依次向上更新祖先节点估计值（例如**获胜次数**和**访问次数**）



- 通过迭代来一步步地扩展博弈树的规模。
- 每一个节点包含**代表的局面**，**被访问的次数**，**累计评分**。

以围棋为例，假设根结点由黑棋行棋，由**UCB1** (Upper Confidence Bound, 上限置信区间) 公式来计算根节点的后续节点如下值，取一个值最大的节点作为后续节点：

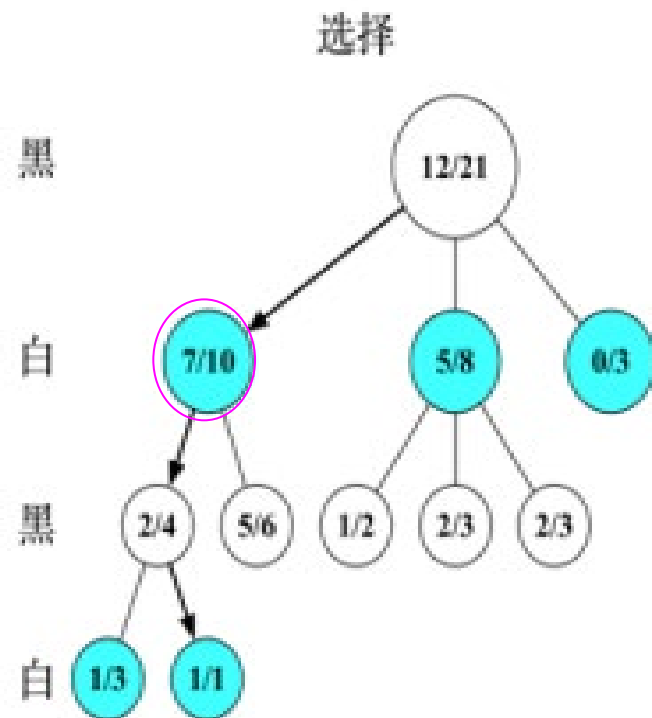
左1：7/10对应的局面奖赏值为

$$\frac{7}{10} + \sqrt{\frac{\ln(21)}{10}} = 1.252$$

第一部分是过去获得胜利的平均次数，

第二部分是未来可能获胜的期望（总次数所带来的机会成本）

UCB1策略：在选择已知回报最多的矿区（**exploitation**，选择表现好的节点）和尝试尽可能多的矿区（**exploration**，尝试未充分探索的节点）之间平衡（**探索和利用的平衡**）



图中每一个节点都代表一个局面，每一个局面记录两个值**A/B**：
A：该局面被访问中黑棋胜利次数。对于黑棋表示己方胜利次数，对于白棋表示己方失败次数（对方胜利次数）；
B：该局面被访问的总次数。

以围棋为例，假设根结点由黑棋行棋，由**UCB1**（Upper Confidence Bound，上限置信区间）公式来计算根节点的后继节点如下值，取一个值最大的节点作为后续节点：

左1：7/10对应的局面奖赏值为

$$\frac{7}{10} + \sqrt{\frac{\ln(21)}{10}} = 1.252$$

左2，5/8对应的局面奖赏值为 $\frac{5}{8} +$

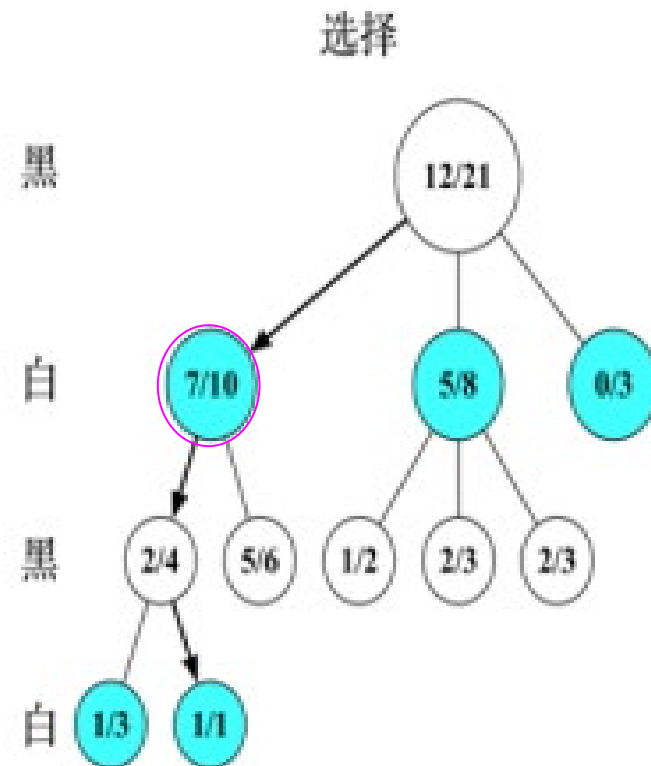
$$\sqrt{\frac{\ln(21)}{8}} = 1.243$$

左3，0/3对应的局面评估分数为

$$\frac{0}{3} + \sqrt{\frac{\ln(21)}{3}} = 1.007$$

由此可见，黑棋会选择局面**7/10**进行行棋。

第一部分是过去获得胜利的平均次数，
第二部分是**未来可能获胜的期望**（总次数所带来的机会成本）



图中每一个节点都代表一个局面，每一个局面记录两个值**A/B**：
A：该局面被访问中**黑棋胜利次数**。对于黑棋表示己方胜利次数，对于白棋表示己方失败次数（对方胜利次数）；
B：该局面被访问的总次数。

在节点7/10，由白棋行棋，评估该节点下面的两个局面，由UCB1公式可得（注意：此时A记录的是白棋失败的次数，所以第一项为 $1-A/B$ ）：

左1，2/4对应的局面奖赏为 $1 - \frac{2}{4} +$

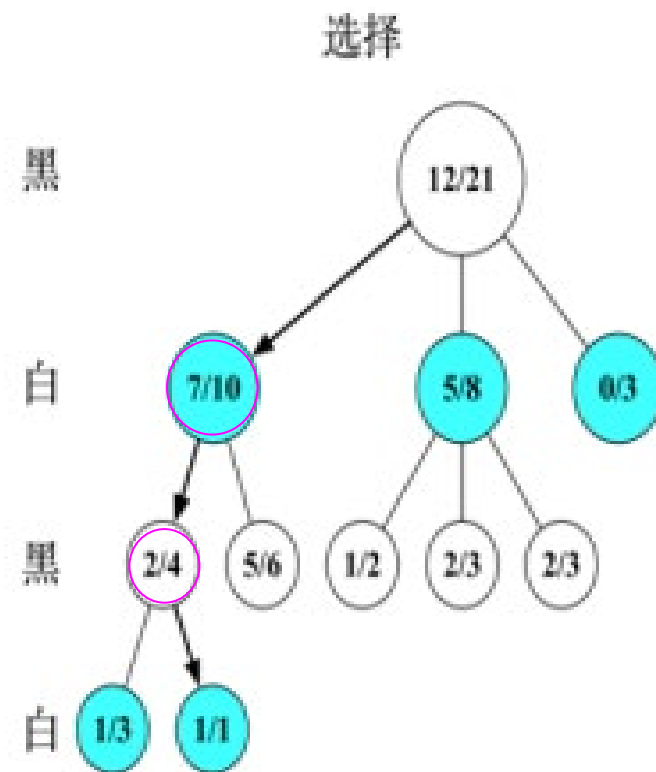
$$\sqrt{\frac{\ln(10)}{4}} = 1.26$$

左2，5/6对应的局面奖赏为 $1 - \frac{5}{6} +$

$$\sqrt{\frac{\ln(10)}{6}} = 0.786$$

由此可见，白棋会选择局面2/4进行行棋。

第一部分是过去获得胜利的平均次数，
第二部分是未来可能获胜的期望（总次数所带来的机会成本）



图中每一个节点都代表一个局面，每一个局面记录两个值A/B：
A：该局面被访问中黑棋胜利次数。对于黑棋表示己方胜利次数，对于白棋表示己方失败次数（对方胜利次数）；
B：该局面被访问的总次数。

在节点2/4，黑棋评估下面的两个局面，由UCB1公式得：

左1，1/3对应的局面奖赏为 $\frac{1}{3} + \sqrt{\frac{\ln(4)}{3}}$:

1.01

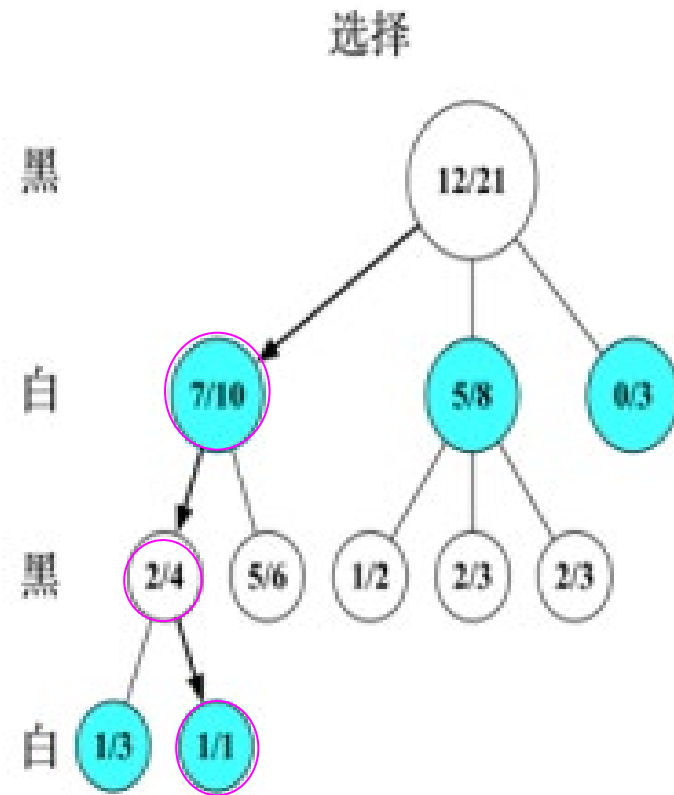
左2，1/1对应的局面奖赏 $\frac{1}{1} + \sqrt{\frac{\ln(4)}{1}} =$

2.18

第一部分是过去获得胜利的平均次数，
第二部分是未来可能获胜的期望（总次数所带来的机会成本）

由此可见，黑棋会选择局面1/1进行行棋。此时已经到达叶子结点，需要进行扩展。

随机生成一个新节点。由于该新节点未被访问，所以初始化为0/0，接着在该节点下进行模拟。

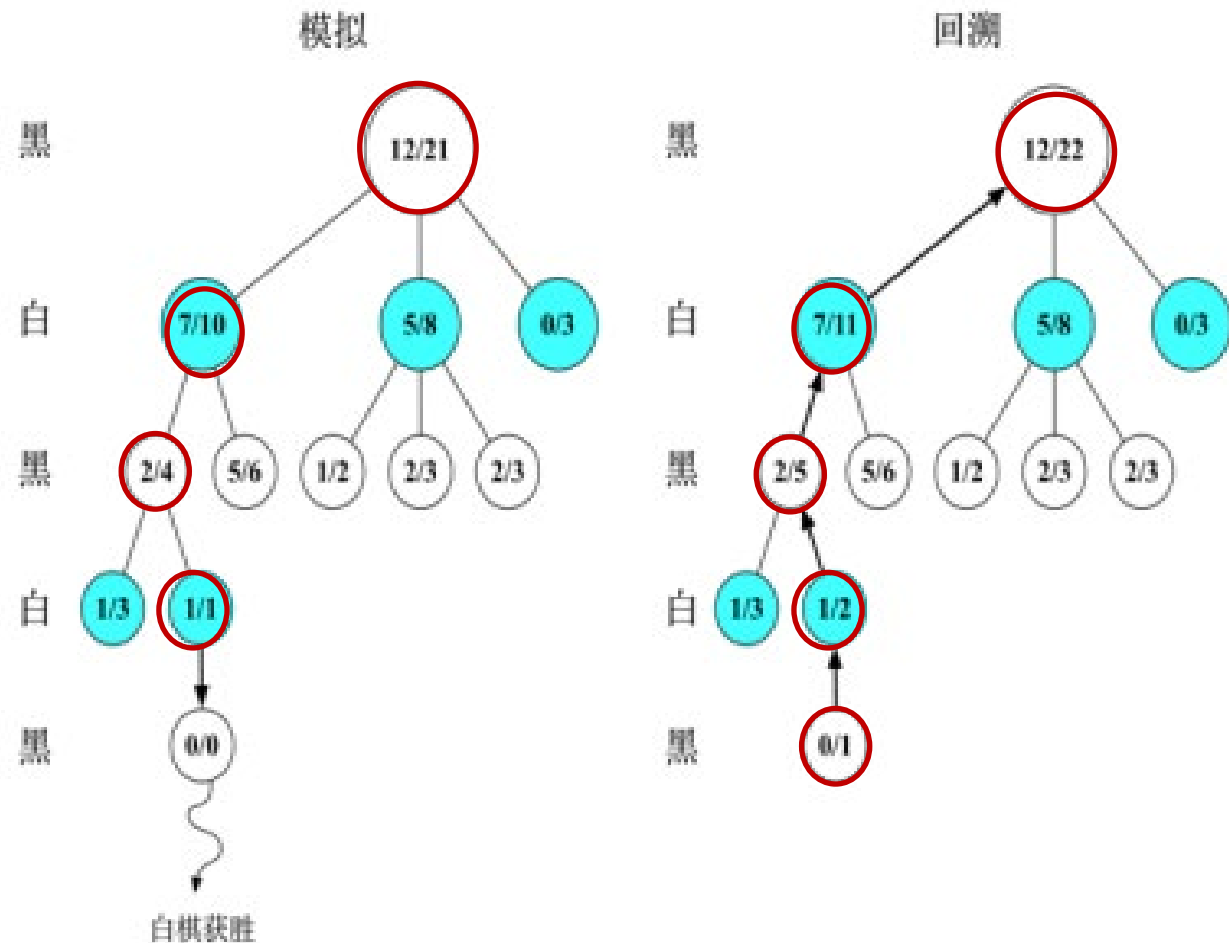


图中每一个节点都代表一个局面，每一个局面记录两个值A/B:
A: 该局面被访问中黑棋胜利次数。对于黑棋表示己方胜利次数，对于白棋表示己方失败次数（对方胜利次数）；
B: 该局面被访问的总次数。

假设经过随机模拟行棋后，最终白棋获胜。

根据模拟结果来更新该模拟路径上每个节点的A/B值，该新节点的A/B值被更新为0/1，并向上回溯到该模拟路径上新节点的所有父辈节点，即所有父辈节点的A不变，B值加1。

蒙特卡洛树搜索基于随机模拟(采样)来得到结果、而非穷尽式枚举（虽然在枚举过程中也可剪掉若干不影响结果的分支）。它结合了树搜索的准确性和随机模拟的效率，尤其适用于那些难以设计启发式函数的复杂问题。



图中每一个节点都代表一个局面，每一个局面记录两个值A/B。
A：该局面被访问中黑棋胜利次数。对于黑棋表示己方胜利次数，对于白棋表示己方失败次数（对方胜利次数）；
B：该局面被访问的总次数。

围棋是完全信息博弈，从理论上来说可以通过暴力搜索所有可能的对弈过程来确定最优的走法

基本算法

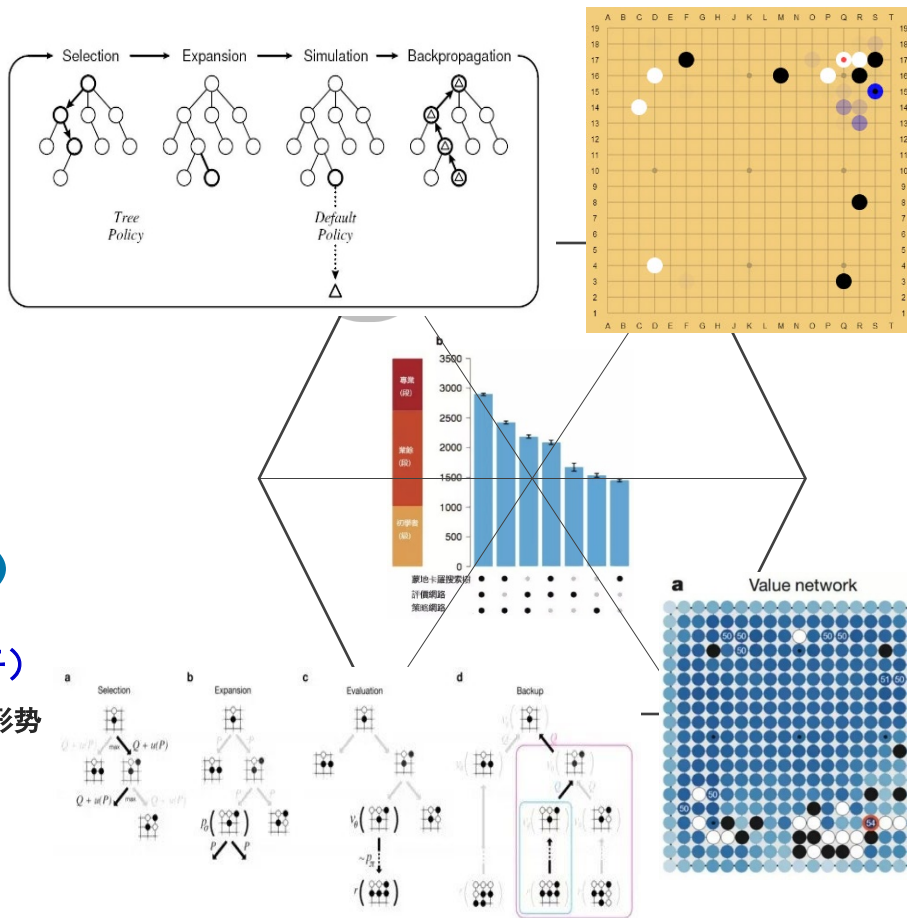
MCTS (蒙特卡洛树搜索)

给胜率高的点分配更多的计算力
任意时间算法，计算越多越精确
1、选择 2、扩展 3、模拟 4、反传

快速模拟

Rollout (随机模拟走子)

通过随机模拟走子胜率来判定形势
速度很快 (1ms/盘)
随机性与合理性的平衡



控制宽度 (250)

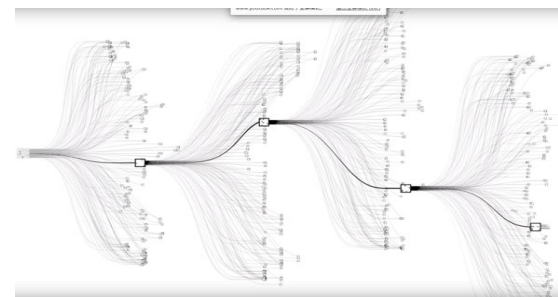
Policy Network (策略网络)

13个卷积层，每层192个卷积核，每个卷积核3*3，参数个数800万+
GPU 3ms/步
预测准确率 57%

控制深度 (150)

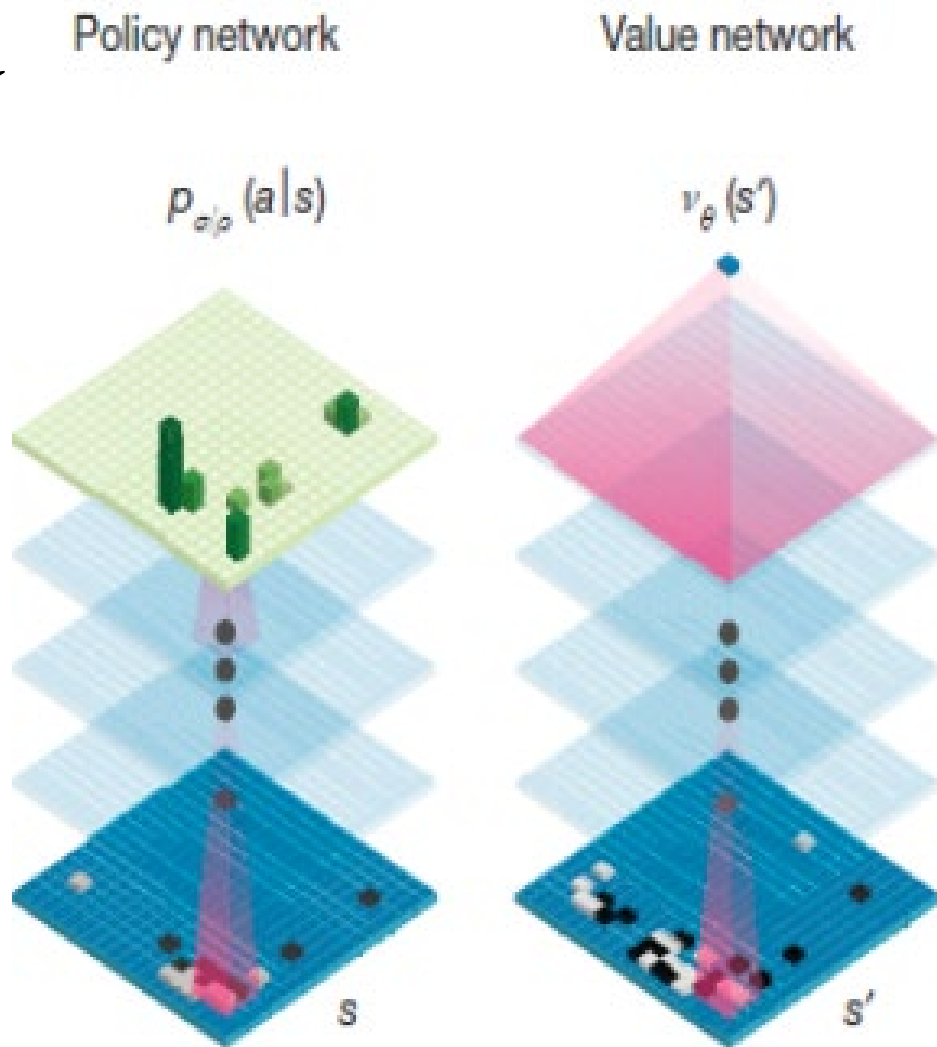
Value Network (价值网络)

在每个分支节点直接判断形势
与Rollout随机模拟相结合，互为补充



用于预测下
一步走法的
概率分布

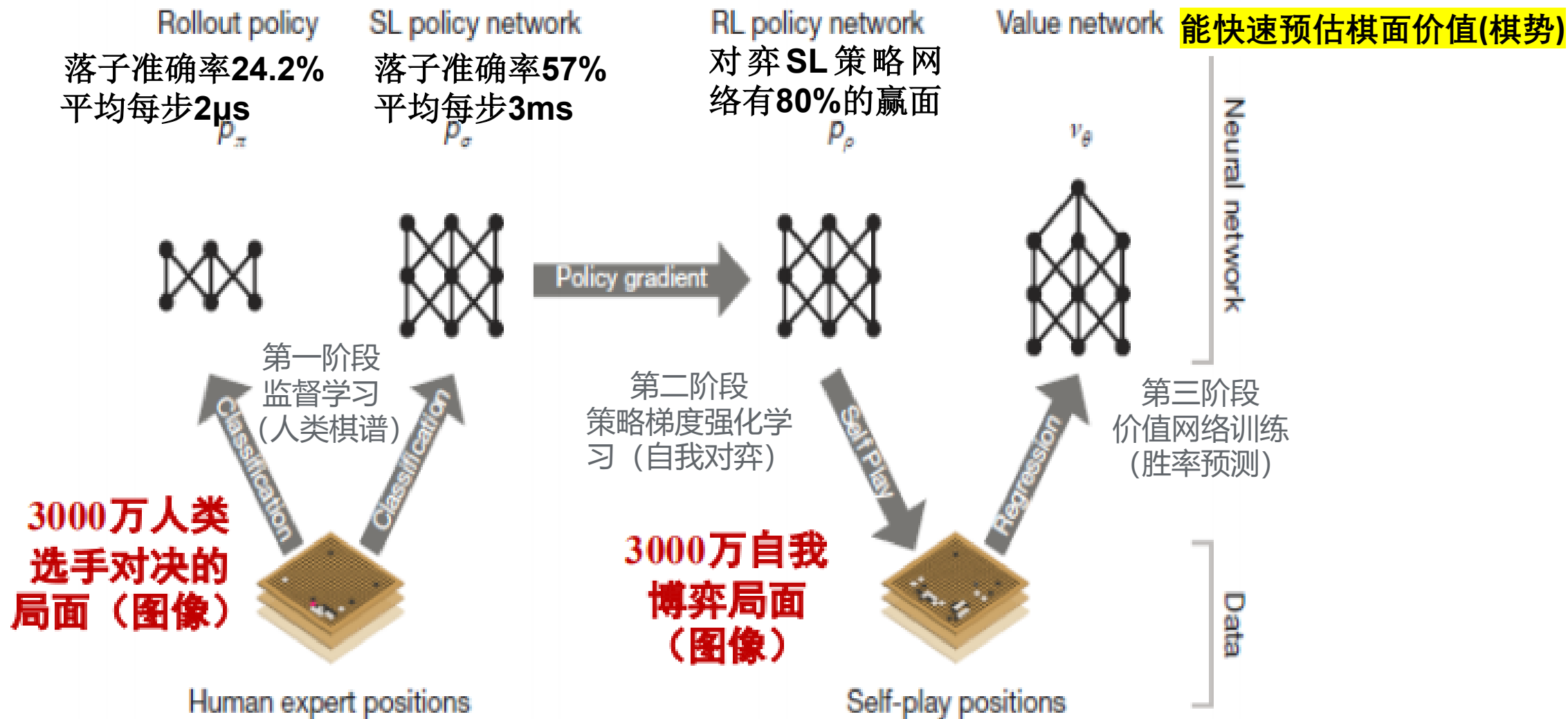
Deepmind 公司在
2016 年 1 月 28 日
Nature 杂志发表
论文《Mastering
the game of Go
with deep neural
networks and
tree search》



用于评估局面
的价值，输出一个
标量，即当前局
面落子后整盘游
戏的胜率

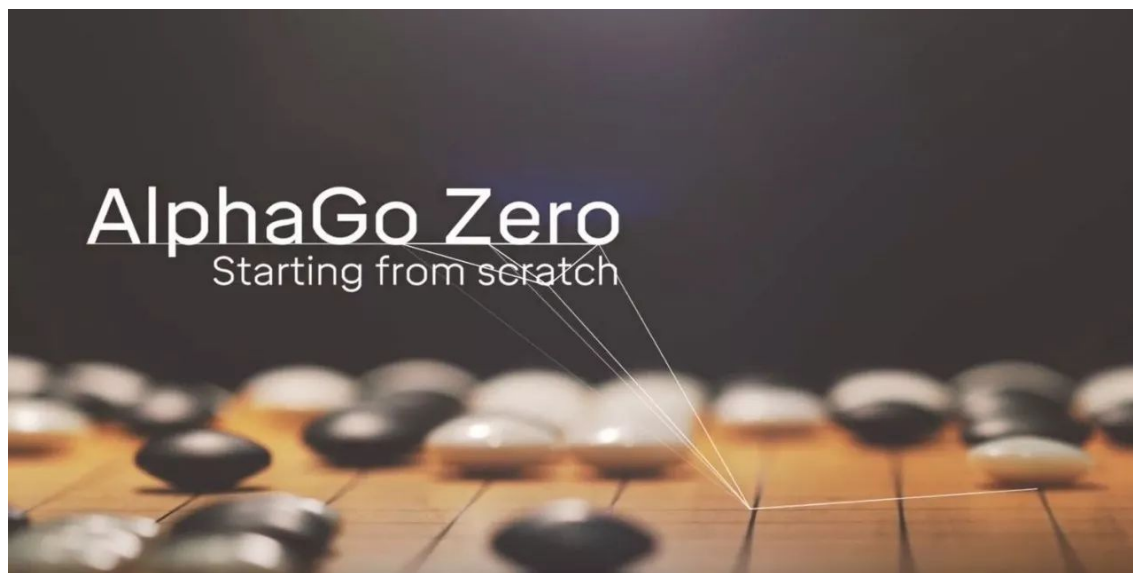
$p_{\sigma,p}(a|s)$ 表示当前状态为 s
(局面)时，采取行动 a
后所得到的概率； $v_{\theta}(s')$
表示当前状态为 s' 时，整
盘棋获胜的概率。

AlphaGo利用了蒙特卡洛树状搜索与两个深度神经网络相结合的方法，以**估值网络value network**来评估大量落子间的优劣，而以**策略网络policy network**来选择落子。

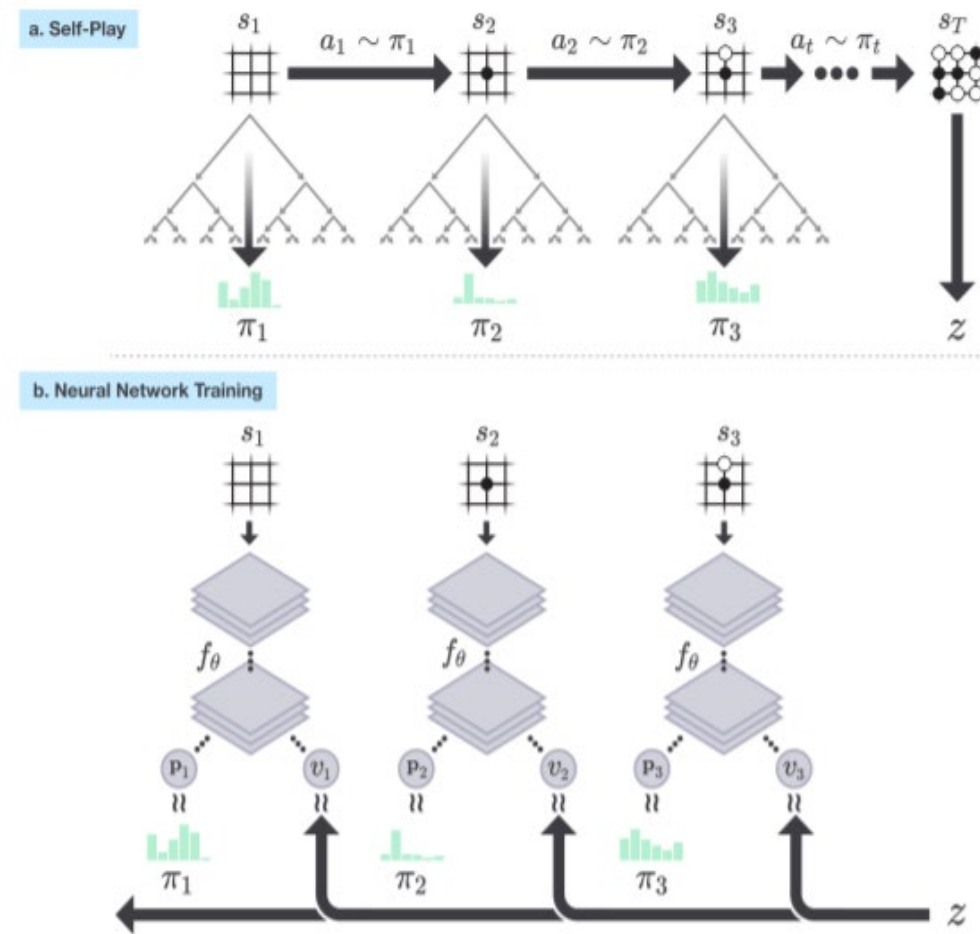


在MCTS的框架下引入两个卷积神经网络policy network和value network以改进纯随机的Monte Carlo模拟，并借助监督学习supervised learning和强化学习reinforcement learning训练这两个网络

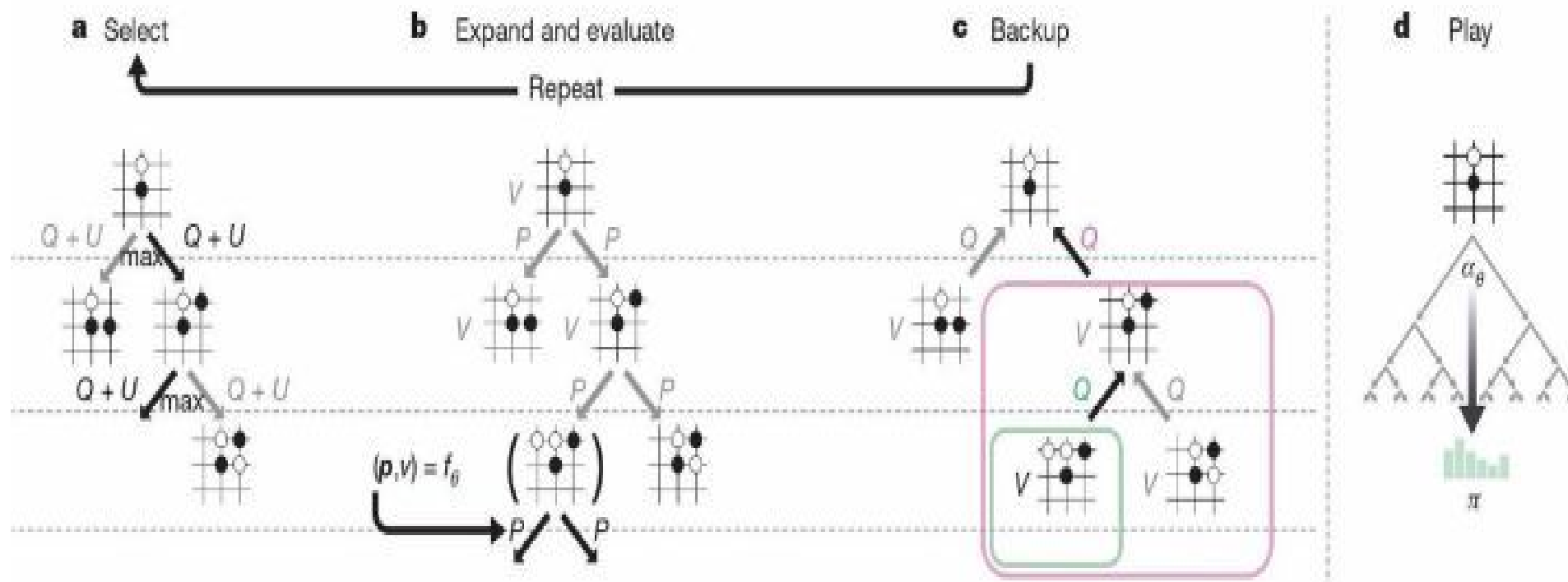
AlphaGo Fan (2015) → AlphaGo Lee (2016) → AlphaGo Master (2017) → AlphaZero (2017)



Mastering the game of Go without human knowledge, Nature, 2017.10



- ◆ 纯自我对弈：不需要人类选手对决的棋面进行训练。
- ◆ 策略网络和价值网络合并，用深度残差网络（ResNet）来改进围棋投子策略，输入是当前的棋面，输出是当前棋面的赢率、下一手投子的位置及其概率。
- ◆ Monte Carlo Tree Search：不穷举所有组合，找到最优或次优位置。置信上限：鼓励探索新的投子位置，越是以往很少投子的位置，得分越高。



蒙特卡洛树搜索中融入了策略网络和价值网络，给定节点 v_n ，将具有如下最大值的节点 v 选择作为 v_0 的后续节点：

动作价值（平均模拟结果）

$$\frac{Q(v)}{N(v)} + \frac{P(v|v_0)}{1 + N(v)}$$

访问次数

- 这里 $P(v|v_0)$ 的值由策略网络计算得到。
- 在模拟策略阶段(default policy)，AlphaGo不仅考虑仿真结果，而且考虑价值网络计算结果。
- 策略网络和价值网络是离线训练得到的。

围棋是完全信息博弈，从理论上来说可以通过暴力搜索所有可能的对弈过程来确定最优的走法

基本算法

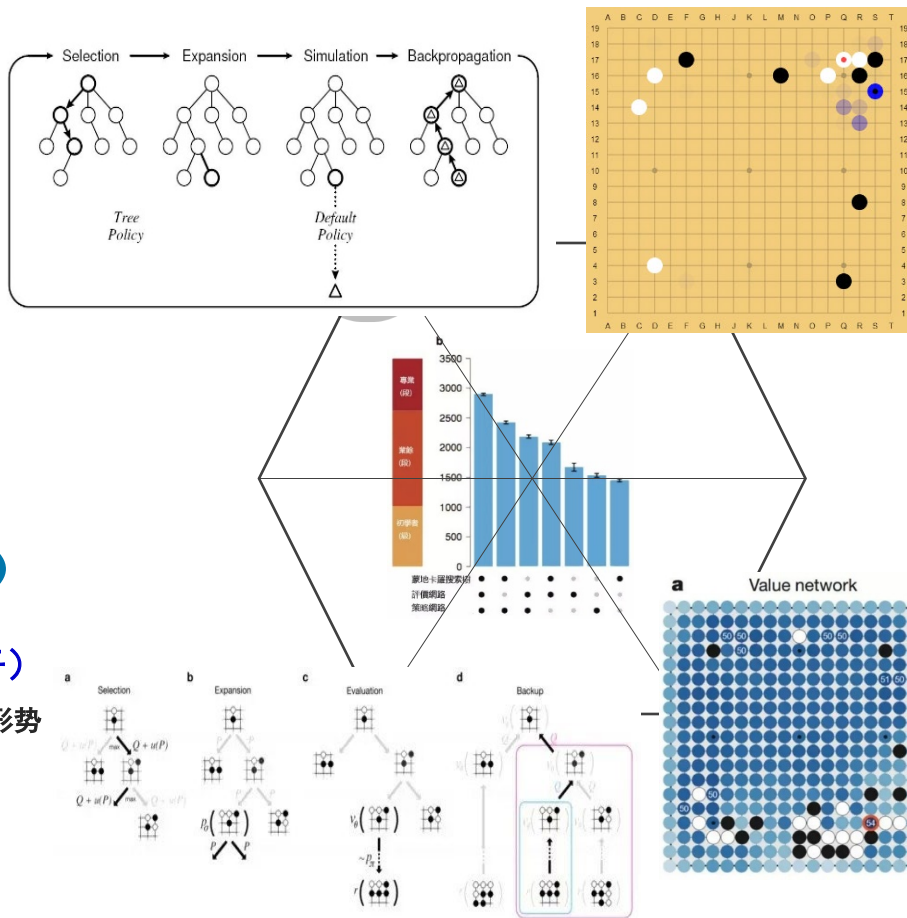
MCTS (蒙特卡洛树搜索)

给胜率高的点分配更多的计算力
任意时间算法，计算越多越精确
1、选择 2、扩展 3、模拟 4、反传

快速模拟

Rollout (随机模拟走子)

通过随机模拟走子胜率来判定形势
速度很快 (1ms/盘)
随机性与合理性的平衡



控制宽度 (250)

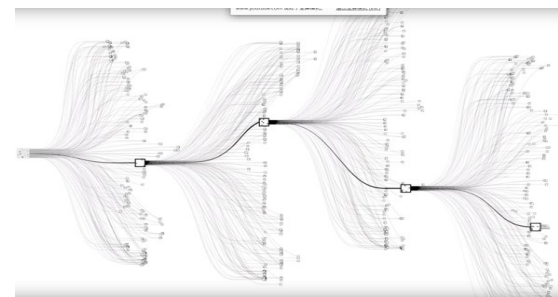
Policy Network (策略网络)

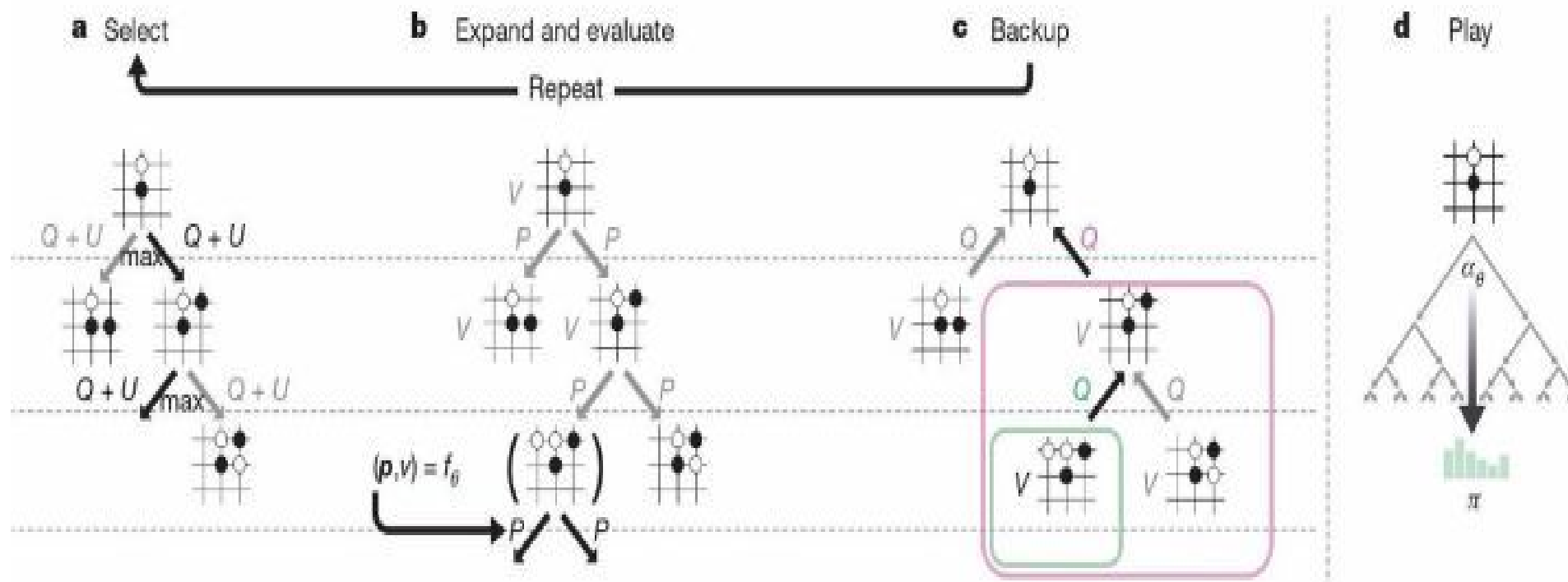
13个卷积层，每层192个卷积核，每个卷积核3*3，参数个数800万+
GPU 3ms/步
预测准确率 57%

控制深度 (150)

Value Network (价值网络)

在每个分支节点直接判断形势
与Rollout随机模拟相结合，互为补充





蒙特卡洛树搜索中融入了策略网络和价值网络，给定节点 v_n ，将具有如下最大值的节点 v 选择作为 v_0 的后续节点：

动作价值（平均模拟结果）

$$\frac{Q(v)}{N(v)} + \frac{P(v|v_0)}{1 + N(v)}$$

访问次数

- 这里 $P(v|v_0)$ 的值由策略网络计算得到。
- 在模拟策略阶段(default policy)，AlphaGo不仅考虑仿真结果，而且考虑价值网络计算结果。
- 策略网络和价值网络是离线训练得到的。

围棋是完全信息博弈，从理论上来说可以通过暴力搜索所有可能的对弈过程来确定最优的走法

基本算法

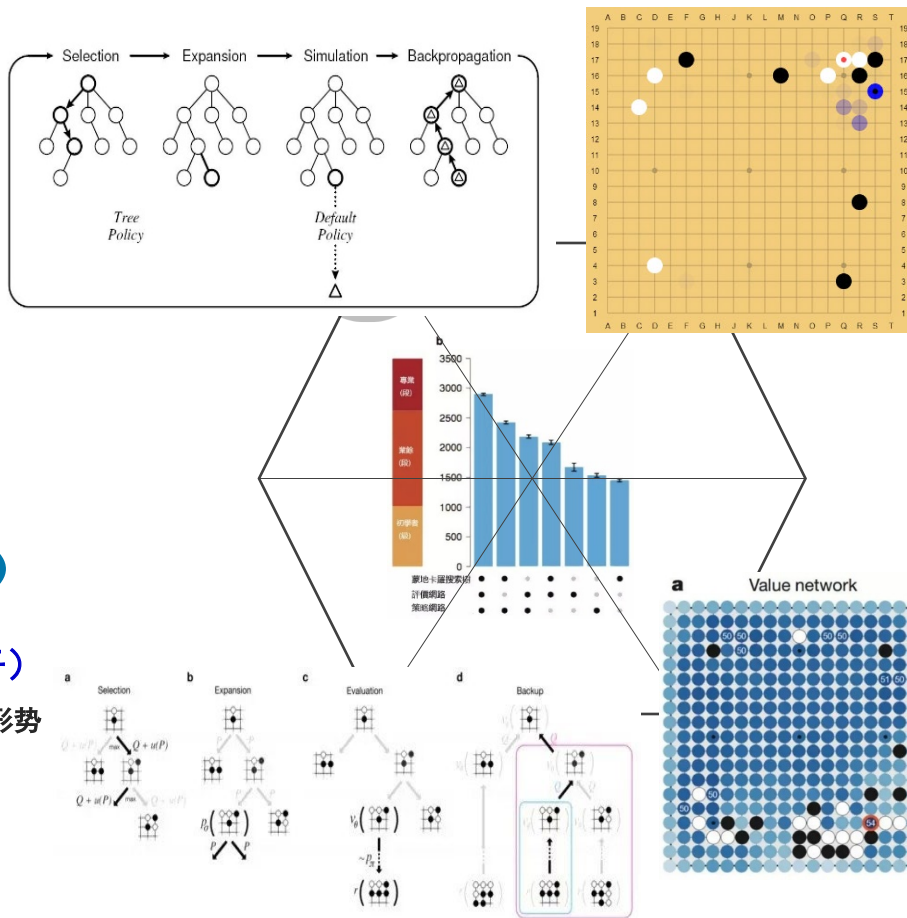
MCTS (蒙特卡洛树搜索)

给胜率高的点分配更多的计算力
任意时间算法，计算越多越精确
1、选择 2、扩展 3、模拟 4、反传

快速模拟

Rollout (随机模拟走子)

通过随机模拟走子胜率来判定形势
速度很快 (1ms/盘)
随机性与合理性的平衡



控制宽度 (250)

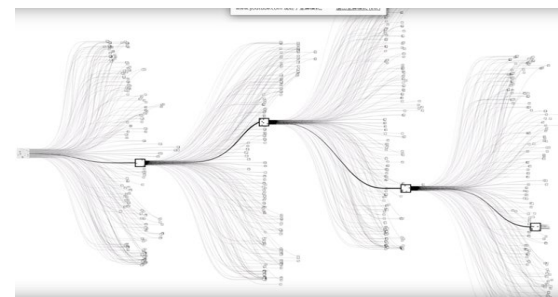
Policy Network (策略网络)

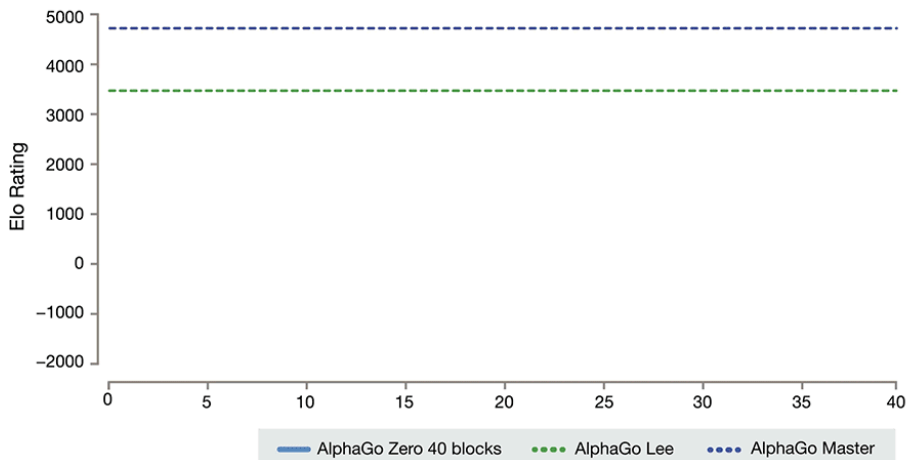
13个卷积层，每层192个卷积核，每个卷积核3*3，参数个数800万+
GPU 3ms/步
预测准确率 57%

控制深度 (150)

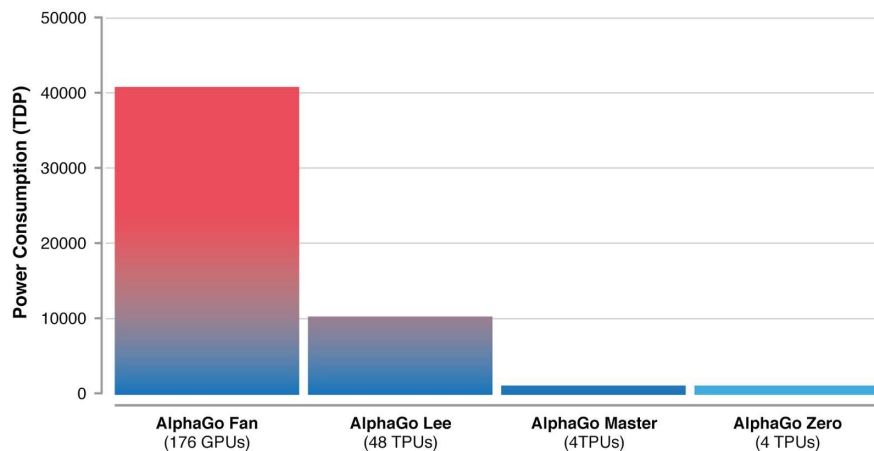
Value Network (价值网络)

在每个分支节点直接判断形势
与Rollout随机模拟相结合，互为补充





从零开始的学习曲线（**ResNet** 由**20**个残差模块组成，**1**个策略头，1个价值头）



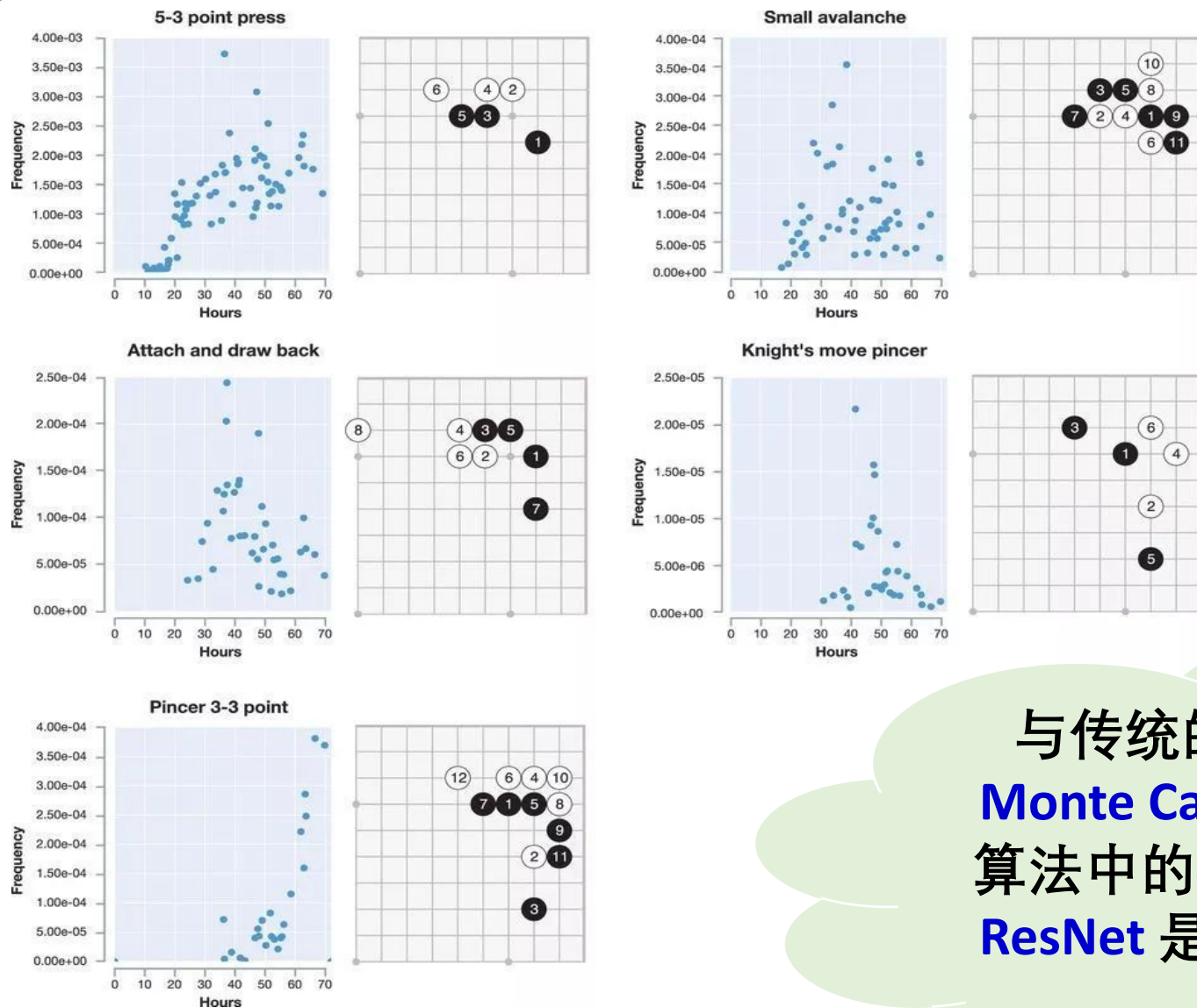
AlphaGo Fan (2015) → AlphaGo Lee (2016) → AlphaGo Master (2017) → AlphaZero (2017)



DeepMind的创始人 CEO Demis Hassabis

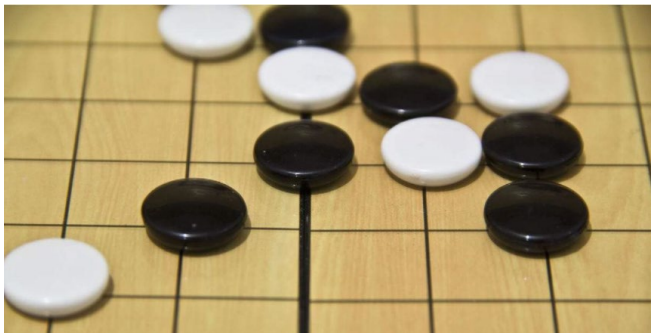
<https://www.ixigua.com/6516785062397084163>

ResNet 由**20**个模块组成，不同训练阶段进行的自我对弈游戏中的头**80**步



其中某些位置被投子的概率，比其它位置显著地高，
 这些位置加上前面几手的
 落子位置和相应的棋面，
 就是围棋定式。

与传统的 A* 算法比较一下，
 Monte Carlo Tree Search 只是 A*
 算法中的树拓展的一种特例，而
 ResNet 是 A* 算法中启发函数的一
 种特例。



AlphaZero实战：从零学下五子棋（附代码）

<https://zhuanlan.zhihu.com/p/32089487>

https://github.com/junxiaosong/AlphaZero_Gomoku

- **6*6的棋盘**：下**4**子棋，大约通过**500~1000**局的**self-play**训练（**2**小时），就能训练出比较靠谱的**AI**；
- **8*8的棋盘**：下**5**子棋，通过大约**2000~3000**局自我对弈训练（**2**天），也能得到比较靠谱的**AI**。

AlphaZero算法本质上是一个最优化搜索算法，对于所有开放信息的离散的最优化问题，只要写出完美的模拟器，就可以应用**AlphaZero**算法。

领域影响：启发后续游戏AI发展（星际争霸、Dota 2），应用于蛋白质折叠（AlphaFold）以及金融、医疗等领域。

训练好的AI模型（3000局self-play训练得到）对弈

AI move: 4,2

Player 1 with X

Player 2 with O

	0	1	2	3	4	5	6	7
7	-	-	-	-	-	-	-	-
6	-	-	-	-	-	-	-	-
5	-	-	-	-	-	-	-	-
4	-	-	X	-	-	-	-	-
3	-	-	-	-	-	-	-	-
2	-	-	-	-	-	-	-	-
1	-	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-	-

每一步棋执行400次MCTS模拟

AI move: 3,5

Player 1 with X

Player 2 with O

	0	1	2	3	4	5	6	7
7	-	-	-	-	-	-	-	-
6	-	-	-	-	-	-	-	-
5	-	-	-	-	-	-	-	-
4	-	-	-	-	-	X	-	-
3	-	-	-	-	O	X	-	-
2	-	-	-	-	-	-	-	-
1	-	-	-	-	-	-	-	-
0	-	-	-	-	-	-	-	-

每一步棋执行800次MCTS模拟